

Uniwersytet WSB Merito w Poznaniu
Wydział Przedsiębiorczości i Innowacji w Warszawie

Bartosz Krawiś
Numer Albumu: 159807

Wykorzystanie sztucznej inteligencji w programowaniu postaci niezależnych w grach komputerowych

Praca inżynierska

Promotor:
Dr Inż. Piotr Bobiński

Kierunek: Informatyka

Specjalność: Front-End Developer

WARSZAWA, 2026

WSB Merito University in Poznań
Faculty of Entrepreneurship and Innovation in Warsaw

Bartosz Krawiś
Student's ID number: 159807

The use of artificial intelligence in non-player character programming in computer games

Engineering thesis

Supervisor:
Ph.D. Eng Piotr Bobiński

Field of study: Information technology (IT)

Specialization: Front-End Developer

WARSAW, 2026

Spis treści

Wstęp.....	4
1 Teoretyczne podstawy wykorzystania sztucznej inteligencji w programowaniu NPC w grach komputerowych	7
1.1 Definicja i rozwój sztucznej inteligencji w grach	8
1.2 Technologie AI stosowane przy tworzeniu NPC	12
1.3 Algorytmy i modele wspomagające zachowania NPC.....	15
1.4 Personalizacja zachowań NPC z wykorzystaniem AI	18
1.5 Etyczne i prawne aspekty wykorzystania AI w grach	21
2 Praktyczne wykorzystanie AI w procesie programowania NPC	26
2.1 Założenia projektowe i rola NPC w grze	27
2.2 Dobór narzędzi AI do tworzenia zachowań NPC	29
2.3 Proces projektowania i implementacji NPC wspomaganych AI	32
2.4 Testowanie i optymalizacja zachowań NPC	36
2.5 Wnioski z realizacji projektu	39
3 Techniki sztucznej inteligencji stosowane w programowaniu NPC	41
3.1 Maszyny stanów (Finite-State Machines)	42
3.2 Drzewa behawioralne (Behavior Trees)	46
3.3 Sieci neuronowe i uczenie maszynowe	49
3.4 Systemy oparte na regułach i logice rozmytej	52
3.5 Planowanie ścieżek i algorytmy nawigacyjne.....	54
4 Zakończenie	57
4.1 Podsumowanie wniosków	57
4.2 Weryfikacja postawionych tez.....	59
BIBLIOGRAFIA	61
NETOGRAFIA.....	61
AKTY NORMATYWNE.....	62

Wstęp

W dobie intensywnego rozwoju gier komputerowych oraz rosnących oczekiwań graczy względem realizmu i złożoności rozgrywki, coraz większą rolę odgrywają systemy oparte na sztucznej inteligencji (AI). Ich zastosowanie w grach komputerowych pozwala nie tylko na tworzenie bardziej dynamicznych i złożonych mechanik rozgrywki, ale także na lepsze projektowanie postaci niezależnych (Non-Playable Characters), które mogą dzięki temu reagować na zachowania gracza w sposób inteligentny i bardziej wiarygodny. Dzięki wykorzystaniu AI postacie te przestają być jedynie tłem lub przeszkodami, a stają się integralną częścią gry, zdolną do podjęcia autonomicznych decyzji, adaptacji do świata gry oraz generowania wrażenia realistycznego zachowania.

Projektowanie NPC z wykorzystaniem sztucznej inteligencji wykracza poza tradycyjne metody, oparte na prostych skryptach lub regułach warunkowych „Jeżeli A to wykonaj B”. Współcześnie twórcy gier stosują różnorodne techniki, jak na przykład maszyny stanów (Finite-State Machines), drzewa behawioralne (Behavior Trees) czy algorytmy planowania akcji (GOAP – Goal-Oriented Action Planning), a czasami również wykorzystywane są elementy uczenia maszynowego (machine learning). Zastosowanie takich rozwiązań umożliwia postaciom NPC reagowanie w czasie rzeczywistym na warunki świata gry, oraz pozwala projektantom tworzyć scenariusze w których postacie niezależne stają się nieprzewidywalne i angażujące dla gracza.

Celem pracy jest zaprojektowanie, implementacja oraz weryfikacja systemu sztucznej inteligencji sterującego zachowaniem postaci niezależnych w dwuwymiarowej grze platformowej zrealizowanej w środowisku Unity. Praca ma charakter inżynierski i łączy część teoretyczną, która dotyczy technologii, algorytmów i modeli stosowanych przy programowaniu postaci NPC, oraz część praktyczną w której zostanie omówione projektowanie, implementacja oraz testowanie postaci niezależnych w środowisku Unity.

W pracy przyjęto następujące tezy badawcze:

1. W grach o niewielkiej skali poprawnie zaprojektowany system oparty na skończonych automatach stanów pozwala uzyskać czytelne, spójne oraz wiarygodne zachowania postaci niezależnych.
2. O powodzeniu implementacji sztucznej inteligencji w grze decyduje nie złożoność zastosowanego algorytmu decyzyjnego, lecz jakość jego integracji z pozostałymi systemami silnika, w szczególności z systemem animacji oraz fizyką.
3. Zastosowanie architektury komponentowej, rozdzielającej logikę decyzyjną od pozostałych funkcji postaci, zwiększa czytelność projektu oraz ułatwia jego testowanie i dalszą rozbudowę.
4. Dobór techniki sterowania zachowaniem postaci niezależnych powinien wynikać ze skali i charakteru produkcji, a nie z dążenia do zastosowania rozwiązań o największej złożoności technicznej.

W pracy przyjęto następujące założenia metodologiczne. Część teoretyczną opracowano metodą analizy literatury przedmiotu, obejmującej publikacje naukowe oraz branżowe poświęcone sztucznej inteligencji w grach komputerowych, a także dokumentację techniczną wykorzystywanego silnika. Część praktyczna ma charakter projektowy i polega na samodzielnym zaprojektowaniu oraz zaimplementowaniu systemu sterowania zachowaniem postaci niezależnych w środowisku Unity z wykorzystaniem języka C#.

Weryfikację przyjętych rozwiązań przeprowadzono metodą badań własnych o charakterze jakościowym, obejmujących testy funkcjonalne zachowań przeciwników, analizę sytuacji granicznych oraz obserwację działania systemu z wykorzystaniem narzędzi diagnostycznych silnika.

Wybór tematu pracy wynika z rosnącego znaczenia sztucznej inteligencji w branży gier komputerowych oraz z osobistego zainteresowania nowoczesnymi technikami wspomagającymi interakcję w świecie wirtualnym. Zastosowanie sztucznej inteligencji w projektowaniu NPC pozwala nie tylko na podniesienie realizmu i atrakcyjności gry, lecz stanowi także istotny element procesu edukacyjnego dla przyszłych programistów i projektantów gier. Praca pozwala zrozumieć mechanizmy sterujące zachowaniem NPC, a także pokazuje wyzwania związane z ich projektowaniem, takie jak na przykład zachowanie równowagi między złożonością a jego wydajnością oraz odpowiedzialnym wykorzystaniem danych gracza. Struktura pracy została dostosowana do logicznego przebiegu zagadnienia. Rozdział 1 obejmuje teoretyczne podstawy sztucznej inteligencji w kontekście NPC – przedstawia definicję AI w grach, opisuje jej rozwój, technologie stosowane przy tworzeniu postaci niezależnych, algorytmy decyzyjne oraz zagadnienia związane z personalizacją. Rozdział 2 przedstawia praktyczne wykorzystanie AI w procesie tworzenia NPC w grze platformowej 2D, w tym założenia projektowe, dobór narzędzi, proces implementacji oraz optymalizacji zachowań postaci. Rozdział 3 koncentruje się na szczegółowym omówieniu technik sztucznej inteligencji stosowanych w programowaniu postaci niezależnych, w tym maszyn stanów, drzew behawioralnych, sieci neuronowych i uczenia maszynowego, systemów regułowych i logiki rozmytej oraz algorytmów planowania ścieżek.

Realizacja pracy pozwala ukazać, w jaki sposób sztuczna inteligencja może wspierać projektowanie realistycznych, angażujących i elastycznych zachowań NPC w grach komputerowych, przy jednoczesnym zachowaniu równowagi między wydajnością systemu a jakością doświadczenia gracza.

1 Teoretyczne podstawy wykorzystania sztucznej inteligencji w programowaniu NPC w grach komputerowych

Sztuczna inteligencja odgrywa coraz istotniejszą rolę w projektowaniu zachowań postaci niezależnych (Non-Playable Characters - NPC) we współczesnych grach komputerowych. Jej zastosowanie nie ogranicza się jedynie do zwiększania ogólnego poziomu rozgrywki, lecz umożliwia tworzenie postaci zdolnych do podejmowania autonomicznych decyzji, reagowania na działanie gracza oraz adaptacji do zmieniających się warunków środowiska gry. Dzięki wykorzystaniu algorytmów AI, NPC przestają być statycznymi postaciami, a zaczynają funkcjonować jako aktywni uczestnicy rozgrywki, dynamicznie reagujący na otoczenie i zachowania użytkownika.

W kontekście programowania NPC sztuczna inteligencja pełni funkcję nadrzędnego systemu decyzyjnego, który można określić jako „mózg” postaci. System ten odpowiada za analizę sytuacji w świecie gry, wybór odpowiednich interakcji oraz reagowanie na obecność i działania gracza. Odpowiednio zaprojektowany i zaimplementowany mechanizm decyzyjny pozwala na zwiększenie spójności zachowań NPC, co w praktyce przekłada się na zwiększenie realizmu rozgrywki oraz satysfakcji użytkownika.

Istotnym aspektem wykorzystania sztucznej inteligencji przy programowaniu NPC jest możliwość modelowania różnorodnych schematów zachowań – od prostych opartych na regułach warunkowych, po bardziej złożone systemy decyzyjne, takie jak maszyny stanów (Finite-State Machine), drzewa behawioralne (Behavior Trees) czy algorytmy planowania (GOAP- Goal-Oriented Action Planning). Zastosowanie tych rozwiązań umożliwia tworzenie postaci zdolnych do dynamicznego reagowania na zmiany w środowisku gry oraz podejmowania decyzji w oparciu o aktualny stan świata wirtualnego.

W zależności od przyjętej technologii oraz charakteru gry, systemy AI mogą odpowiadać za sterowanie ruchem postaci, wybór strategii działania, reagowanie na obecność gracza, a także realizację zaprogramowanych celów. Dobór odpowiednich technik ma kluczowe

znaczenie dla uzyskania spójnych i wiarygodnych zachowań NPC, które pozostają czytelne dla użytkownika, a jednocześnie wystarczająco złożone, aby zapewnić atrakcyjność rozgrywki.

1.1 Definicja i rozwój sztucznej inteligencji w grach

Sztuczna inteligencja w grach komputerowych jest najczęściej rozumiana jako zbiór metod i technik, które umożliwiają symulowanie zachowań uznawanych przez gracza za „inteligentne” w ramach świata gry. W praktyce oznacza to projektowanie takich mechanizmów decyzyjnych, które pozwalają postaciom niezależnym – NPC analizować stan otoczenia, wybierać działania oraz reagować na zachowania gracza w sposób spójny i wiarygodny. W odróżnieniu od sztucznej inteligencji w sensie ogólnym (wykorzystywanej w celach badawczo-naukowych), której celem jest tworzenie systemów zdolnych do uczenia się, rozumowania i pozyskiwania wiedzy w szerokim zakresie problemów, AI w grach jest zwykle podporządkowane celowi projektowemu: ma zapewnić określony poziom wyzwania, realizmu i przewidywalności oraz wspierać atrakcyjność i dramaturgię rozgrywki.^{1 2}

Warto podkreślić, że w grach „inteligencja” nie jest oceniana wyłącznie przez pryzmat optymalności decyzji. Postać niezależna, która zawsze podejmuje najlepszą możliwą decyzję, może w praktyce obniżyć jakość doświadczenia gracza, ponieważ stanie się frustrująca lub „niesprawiedliwa” z perspektywy gracza. Z tego względu projektanci często świadomie stosują uproszczenia, ograniczenia percepcji NPC (np. ograniczony zasięg widzenia, opóźnienie reakcji) oraz elementy losowości. W efekcie powstaje AI, które nie musi być „najsilniejsze” ale powinno być czytelne, wiarygodne, sprawiedliwe oraz spójne z zasadami świata gry.³

Historyczny rozwój sztucznej inteligencji w grach prowadził od prostych reguł do rozbudowanych systemów decyzyjnych, a jego przebieg był ściśle związany z możliwościami

¹ I. Millington, J. Funge, *Artificial Intelligence for Games*, CRC Press, Boca Raton 2018.

² G. N. Yannakakis, J. Togelius, *Artificial Intelligence and Games*, Springer, Cham 2018.

³ I. Millington, J. Funge, dz. cyt.

ówczesnego sprzętu i rosnącą złożonością samych produkcji. W początkowych etapach — szczególnie w erze gier arcade — zachowania przeciwników realizowane były w sposób mocno deterministyczny. Dominowały proste reguły typu „jeżeli A, to B” oraz zachowania oparte na zaprogramowanych schematach. Wynikało to z ograniczeń mocy obliczeniowej i pamięci maszyn oraz z prostoty ówczesnych mechanik rozgrywki. Mimo to nawet takie podejście potrafiło tworzyć iluzję inteligencji: gracz obserwował reakcję na swoje działania i interpretował ją jako celową, choć była ona wynikiem kilku prostych instrukcji.⁴

Wraz z upowszechnieniem gier szerzej rozbudowanych (zarówno pod kątem świata gry jak i liczby możliwych sytuacji), proste reguły przestały być wystarczające. Pojawiła się potrzeba modelowania zachowań w sposób bardziej uporządkowany, co doprowadziło do popularyzacji struktur takich jak skończone automaty stanów (Finite-State Machine). Dzięki FSM możliwe stało się projektowanie zachowań NPC jako przełączania się pomiędzy stanami (np. „pościg”, „idle (stanie w miejscu)”, „atak”, „ucieczka”) w zależności od warunków otoczenia i zachowania gracza. Klasycznym przykładem jest gra *Pac-Man* (1980), w której każdy z czterech duchów działa jako prosty automat przełączający się między kilkoma stanami, a jego „osobowość” wynika z odmiennej reguły wyboru celu w pościgu. Mimo trywialnej logiki gracz odczytuje duchy jako cztery różne charaktery — co dobrze ilustruje, że w grach liczy się wrażenie inteligencji, a nie jej rzeczywista złożoność. Taka formalizacja ułatwiła tworzenie zachowań, testowanie ich oraz rozbudowę, a jednocześnie pozostała stosunkowo lekka obliczeniowo.^{5 6 7}

Kolejnym istotnym kierunkiem rozwoju było coraz szersze stosowanie algorytmów nawigacyjnych i wyszukiwania ścieżek. W grach, w których NPC poruszały się po większych mapach (np. gry Open World) lub musiały omijać przeszkody, kluczowe stało się planowanie ruchu. Algorytm A* (A-star) oraz pokrewne metody wyszukiwania optymalnej ścieżki stały się standardem w wyznaczaniu tras w środowiskach dyskretnych (siatki, grafy), a w grach 3D ważnym krokiem było rozpowszechnienie siatek nawigacyjnych (navmesh). W przypadku gier

⁴ I. Millington, J. Funge, dz. cyt.

⁵ I. Millington, J. Funge, dz. cyt.

⁶ M. Buckland, *Programming Game AI by Example*, Wordware Publishing, Plano 2005.

⁷ J. Pittman, *The Pac-Man Dossier*, wersja 1.0.27, 2015, <https://pacman.holenet.info/>, [dostęp: 20.07.2026].

2D znaczenie mają klasyczne podejścia grafowe, ale także uproszczone modele nawigacji oparte na detekcji przeszkód, punktach kontrolnych czy regułach ruchu.⁸

W miarę rozwoju produkcji gier rosta też potrzeba tworzenia zachowań bardziej modułowych, możliwych do łatwego rozwijania bez gwałtownego wzrostu złożoności. W tym kontekście coraz większą popularność zaczęły zdobywać drzewa behawioralne (Behavior Trees). Umożliwiają one opis logiki NPC w sposób hierarchiczny, co ułatwia ponowne wykorzystanie elementów zachowania, testowanie oraz współpracę projektantów z programistami. Drzewa behawioralne stały się w branży jednym z najczęściej wykorzystywanych wzorców, zwłaszcza w przypadku przeciwników o bardziej złożonych rolach.⁹

Równolegle rozwijały się również podejścia oparte na planowaniu celów (np. GOAP – Goal Oriented Action Planning). Na szerszą skalę zastosowano je po raz pierwszy w grze *F.E.A.R.* (2005), w której zamiast odtwarzać sztywny skrypt jak przy FSM, przeciwnicy dynamicznie mogli dopasowywać sekwencję działań, takich jak szukanie osłony lub oskrzydlenie gracza. Rozwiązanie to spopularyzował Jeff Orkin, a podejście do dziś bywa wykorzystywane w projektowaniu adaptacyjnych zachowań NPC. Takie rozwiązanie bywa szczególnie użyteczne, gdy NPC ma wiele możliwych sposobów realizacji celu lub działa w środowisku o dużej zmienności.¹⁰

W nowoczesnych grach AI jest coraz częściej traktowane jako narzędzie wspierające cele projektowe, a nie jako autonomiczny „mózg”. Oznacza to, że projektanci i programiści dążą do uzyskania równowagi pomiędzy:

- Wiarygodnością (NPC jest spójne z rolą i światem gry)
- Czytelnością (gracz jest w stanie zrozumieć, dlaczego NPC tak reaguje)
- Wyzwaniem (NPC stanowi interesującą przeszkodę lub rolę partnera gracza)

⁸ M. Buckland, dz. cyt., rozdz. 5 „The Secret Life of Graphs”; I. Millington, J. Funge, dz. cyt.

⁹ I. Millington, J. Funge, dz. cyt.

¹⁰ J. Orkin, *Three States and a Plan: The AI of F.E.A.R.*, Game Developers Conference, San Francisco 2006; M. Buckland, dz. cyt.

- Wydajnością (zachowanie działa stabilnie w czasie rzeczywistym)
- Kontrolą projektową (twórcy potrafią przewidzieć skutki zmian w AI).¹¹

Rozwój AI w grach to w dużej mierze rozwój metod tworzenia „przekonującej iluzji”. Przykładowo, częstym rozwiązaniem jest ograniczanie percepcji NPC (np. stożek widzenia), modelowanie pamięci krótkotrwałej (NPC „zapomina” po pewnym czasie interakcje z graczem), czy wprowadzenie opóźnień reakcji. Takie mechanizmy sprawiają, że przeciwnik wydaje się bardziej ludzki, a gra pozostaje uczciwa dla użytkownika.¹²

W ostatnich latach w dyskusjach o AI często pojawia się uczenie maszynowe, w tym sieci neuronowe. Warto jednak zaznaczyć, że w praktyce produkcyjnej klasyczne techniki (FSM, Behavior Trees, systemy regułowe) nadal dominują w wielu gatunkach, ponieważ zapewniają przewidywalność oraz łatwość kontroli. Uczenie maszynowe wykorzystywane jest głównie tam, gdzie opłaca się adaptacja, analiza danych lub generowanie zachowań mniej przewidywalnych. Wiąże się to również z wyzwaniami: trudniejszą interpretacją decyzji modelu, większym kosztem testowania oraz ryzykiem zachowań niepożądanych z punktu widzenia balansu gry.^{13 14}

Podsumowując, sztuczna inteligencja w grach komputerowych jest zbiorem metod służących tworzeniu wiarygodnych i użytecznych zachowań w świecie gry, szczególnie w kontekście NPC. Jej rozwój rozpoczął się od prostych reguł i skryptów „Jeżeli-wtedy”, przebiegał przez modele decyzyjne (FSM), aż po te bardziej elastyczne i modularne struktury (Behavior Trees, GOAP) oraz czasami machine learning. Dalsze podrozdziały koncentrują się na technologiach oraz algorytmach najczęściej wykorzystywanych do projektowania i implementacji zachowań postaci niezależnych.

¹¹ I. Millington, J. Funge, dz. cyt.

¹² I. Millington, J. Funge, dz. cyt.

¹³ G. N. Yannakakis, J. Togelius, dz. cyt.

¹⁴ I. Millington, J. Funge, dz. cyt.

1.2 Technologie AI stosowane przy tworzeniu NPC

Rozwój sztucznej inteligencji w grach komputerowych był możliwy nie tylko dzięki postępowi algorytmicznemu, lecz również dzięki powstaniu i doskonaleniu wyspecjalizowanych technologii wspierających projektowanie oraz implementację zachowań postaci niezależnych. Współczesne silniki gier oraz narzędzia deweloperskie oferują rozbudowane środowiska umożliwiające integrację systemów decyzyjnych, nawigacyjnych oraz percepcyjnych w ramach jednego spójnego ekosystemu programistycznego. Technologie te stanowią warstwę pośrednią pomiędzy abstrakcyjnymi modelami sztucznej inteligencji a konkretną implementacją zachowań w świecie gry.¹⁵

Jednym z podstawowych elementów technologicznych wykorzystywanych przy tworzeniu NPC są silniki gier (game engines), które dostarczają gotowe mechanizmy obsługi logiki, fizyki, animacji oraz interakcji ze środowiskiem. Silniki takie jak Unity czy Unreal Engine udostępniają bogate interfejsy programistyczne (API), umożliwiające implementację systemów AI w sposób modularny i łatwy do integracji z pozostałymi komponentami gry. W ramach tych środowisk programista może definiować logikę zachowań NPC, zarządzać ich stanami, reagować na zdarzenia w świecie gry oraz kontrolować przepływ informacji pomiędzy poszczególnymi systemami.^{16 17}

Istotnym obszarem technologii AI są systemy percepcji, które umożliwiają postaciom niezależnym pozyskiwanie informacji o otoczeniu. W praktyce realizowane jest to poprzez mechanizmy detekcji kolizji, rzucania promieni (raycasting) oraz systemy wykrywania dźwięku. Dzięki nim postać może „zauważyć” obecność gracza, przeszkód lub innych obiektów oraz uwzględnić te informacje w procesie decyzyjnym. W projekcie towarzyszącym pracy percepcję przeciwników oparto na progach odległości od gracza; w przypadku nieruchomego przeciwnika typu „Guard” dodatkowo sprawdzany jest kierunek zwrotu postaci. Raycasting wykorzystano natomiast do wykrywania krawędzi platform podczas patrolu.¹⁸

¹⁵ G. N. Yannakakis, J. Togelius, dz. cyt.

¹⁶ I. Millington, J. Funge, dz. cyt.

¹⁷ Unity Technologies, Unity Manual — Navigation and Pathfinding, <https://docs.unity3d.com/Manual/Navigation.html>, [dostęp: 07.08.2026].

¹⁸ M. Buckland, dz. cyt.

Kolejny obszar technologiczny stanowią systemy nawigacyjne. Współczesne silniki oferują gotowe rozwiązania w tym zakresie – Unity udostępnia wbudowany system NavMesh z komponentami agenta, dzięki czemu nie trzeba implementować własnych algorytmów wyszukiwania ścieżek. W przypadku prostych gier 2D rozwiązania te są często zbędne, a ruch postaci można oprzeć na prostszych mechanizmach – w zrealizowanej platformówce przeciwnicy poruszają się po trajektoriach patrolu ograniczonych zasięgiem, z detekcją krawędzi platform, bez wyznaczonych ścieżek.^{19 20}

Równie istotną technologię stanowią narzędzia wspierające projektowanie logiki zachowania na poziomie wizualnym. Unreal Engine oferuje wbudowany edytor drzew behawioralnych, a w Unity taką rolę pełni Animator – graficzny edytor maszyny stanów, wykorzystywany głównie do sterowania animacją oraz płynnymi przejściami pomiędzy stanami. Takie rozwiązania ułatwiają współpracę programistów i projektantów gier, pozwalając na szybsze prototypowanie, testowanie oraz modyfikowanie zachowań NPC. W ramach części praktycznej Animator steruje stanami animacji przeciwników na podstawie parametrów ze skryptu logiki, a zdarzenia animacji (Animation Events) wywołują metody zadające obrażenia – synchronizując moment trafienia z klatką animacji ataku.^{21 22}

W kontekście technologii AI coraz większe znaczenie zyskują również systemy zarządzania zdarzeniami oraz architektury oparte na komponentach. Dzięki nim zachowania NPC mogą być budowane jako zestaw niezależnych modułów, w którym każdy odpowiada za konkretną funkcję taką jak np. percepcja, planowanie, ruch czy interakcja z graczem. Takie podejście zwiększa elastyczność systemu, umożliwia ponowne wykorzystanie kodu oraz ułatwia rozbudowę zachowań w kolejnych etapach produkcji. Architektury komponentowe sprzyjają również testowaniu i debugowaniu, ponieważ pozwalają izolować wybrane elementy logiki. Na tej zasadzie zbudowano przeciwników w części praktycznej: logika stanów, obsługa

¹⁹ I. Millington, J. Funge, dz. cyt.

²⁰ M. Buckland, dz. cyt.

²¹ I. Millington, J. Funge, dz. cyt.

²² Unity Technologies, *Unity Manual — Animator Controller*, <https://docs.unity3d.com/2022.3/Documentation/Manual/class-AnimatorController.html>, [dostęp: 10.08.2026].

zdrowia oraz interfejs paska życia stanowią trzy niezależne komponenty, które komunikują się poprzez zdarzenia, a nie bezpośrednio odwołania. Dzięki takiemu rozwiązaniu jeden komponent zdrowia obsługuje oba typy przeciwników, a warstwa interfejsu jedynie subskrybuje zdarzenie zmiany zdrowia, pozostając niezależna od logiki stanów.²³

W ostatnich latach do zestawu technologii wykorzystywanych przy tworzeniu postaci niezależnych dołączają również narzędzia wspierające integrację uczenia maszynowego. Obejmują one biblioteki do trenowania modeli – takie jak pakiet Unity ML-Agents, który pozwala trenować agentów bezpośrednio w środowisku silnika – a także systemy zapisu i analizy danych rozgrywki łącznie z mechanizmami wdrażania wytrenowanych modeli do gry. Takie rozwiązania nie są jeszcze powszechnie stosowane w produkcjach, stanowią jednak obiecujący kierunek rozwoju, szczególnie w kontekście automatycznego dopasowywania poziomu trudności oraz analizy stylu gry użytkownika. W praktycznej części pracy świadomie zrezygnowano z tego rozwiązania na rzecz technik klasycznych, które przy niewielkiej skali projektu zapewniają przewidywalność oraz prostotę testowania.^{24 25}

Podsumowując, o doborze technologii AI decyduje przede wszystkim skala i charakter produkcji, a nie sama dostępność narzędzi. Rozbudowane systemy nawigacji czy edytory drzew behawioralnych znajdują uzasadnienie w dużych grach o złożonych światach, podczas gdy w prostszych produkcjach — jak prototyp gry zrealizowany w części praktycznej — wystarczające okazują się podstawowe mechanizmy silnika: detekcja kolizji, proste progi percepcji oraz maszyny stanów.

²³ I. Millington, J. Funge, dz. cyt.

²⁴ G. N. Yannakakis, J. Togelius, dz. cyt.

²⁵ Unity Technologies, Unity ML-Agents Toolkit Documentation, <https://unity-technologies.github.io/ml-agents/>, [dostęp: 10.08.2026].

1.3 Algorytmy i modele wspomagające zachowania NPC

Modele decyzyjne stosowane w programowaniu NPC można uporządkować według jednego kryterium: jaką część zachowania postaci projektant definiuje z góry, a jak wiele pozostawia systemowi do rozstrzygnięcia w trakcie rozgrywki. Na jednym końcu tej skali znajdują się rozwiązania w pełni jawne — systemy regułowe i maszyny stanów, w których każda reakcja została przewidziana przez twórcę. Na drugim leżą modele oparte na uczeniu maszynowym, gdzie zachowanie wyłania się z danych treningowych, a projektant oddaje część kontroli i przewidywalności w zamian za większą elastyczność. Pomiędzy tymi biegunami mieszczą się drzewa behawioralne oraz planowanie działań (GOAP), które łączą jawną strukturę ze swobodą doboru akcji.

Jednym z najstarszych stosowanych modeli decyzyjnych są maszyny stanów (Finite-State Machines). W tym podejściu zachowanie postaci opisywane jest jako zbiór stanów oraz przejść pomiędzy nimi, zależnie od spełnienia określonych warunków. Każdy stan reprezentuje konkretny tryb działania NPC, takie jak bezczynność, patrolowanie, pościg czy atak. Przejścia między stanami są inicjowane w odpowiedzi na zdarzenia w świecie gry lub zmiany w stanie wewnętrznym postaci. Prostota tego modelu sprawia, że jest on łatwy do implementacji, testowania oraz utrzymania, co czyni go szczególnie popularnym w grach o umiarkowanej złożoności zachowań. Model ten zastosowany został również w praktycznej części pracy, w której dwa typy przeciwników działają w oparciu o odrębne automaty stanów.

Wraz ze wzrostem skali projektów oraz liczby możliwych sytuacji, klasyczne maszyny stanów zaczęły ujawniać swoje ograniczenia, przede wszystkim w postaci trudności w zarządzaniu dużą liczbą stanów oraz przejść. Odpowiedzią na te problemy stały się hierarchiczne maszyny stanów oraz inne struktury umożliwiające bardziej modularne opisywanie zachowań. Takie podejście pozwala grupować stany w większe jednostki logiczne oraz upraszczać strukturę przejść, co zwiększa czytelność projektu oraz ułatwia jego dalszy rozwój.

Jednym z najważniejszych współczesnych modeli decyzyjnych są drzewa behawioralne (Behavior Trees). W tym podejściu logika zachowania opisywana jest w postaci hierarchicznej

struktury drzewiastej, składającej się z węzłów sterujących oraz węzłów wykonawczych. Węzły sterujące odpowiadają za wybór odpowiedniej gałęzi drzewa w zależności od spełnienia warunków, natomiast węzły wykonawcze realizują konkretne akcje lub sprawdzają stan świata gry. Drzewa behawioralne umożliwiają tworzenie zachowań w sposób modularny oraz łatwy do modyfikacji, co czyni je szczególnie przydatnymi w przypadku postaci o złożonych rolach oraz wielu możliwych trybach działania.

Istotną zaletą drzew behawioralnych jest ich czytelność oraz możliwość wizualnej reprezentacji struktury decyzji. Dzięki temu projektanci gier mogą w prosty sposób analizować przebieg logiki, identyfikować potencjalne błędy oraz wprowadzać zmiany bez konieczności ingerencji w dużą liczbę fragmentów kodu. W praktyce drzewa behawioralne stały się jednym z najczęściej wykorzystywanych wzorców w nowoczesnych produkcjach, zwłaszcza w grach wymagających precyzyjnej kontroli nad zachowaniem przeciwników oraz postaci wspierających gracza.

Kolejną grupę modeli stanowią systemy oparte na regułach decyzyjnych oraz logice warunkowej. W tym podejściu zachowanie postaci niezależnych definiowane jest za pomocą zestawu reguł typu „jeżeli A wtedy B”, które określają reakcje postaci na określone zdarzenia lub stany otoczenia. Systemy regułowe są szczególnie użyteczne w przypadku prostych zachowań, gdzie liczba możliwych interakcji jest ograniczona, a zależności pomiędzy nimi łatwe do opisanie. Ich największą zaletą jest przejrzystość oraz łatwość implementacji, jednak wraz ze wzrostem liczby warunków mogą prowadzić do powstania rozbudowanych zbiorów reguł, trudnych do utrzymania.

W bardziej złożonych systemach decyzyjnych często stosuje się podejścia oparte na planowaniu działań, takie jak Goal-Oriented Action Planning (GOAP). W tym modelu zachowanie NPC nie jest opisane bezpośrednio jako sekwencja stanów, lecz jako zbiór celów, które postać stara się osiągnąć przy wykorzystaniu dostępnych akcji. Każda akcja posiada określone warunki początkowe oraz skutki, a system planowania dobiera sekwencję działań prowadzącą do realizacji wybranego celu. Takie podejście umożliwia tworzenie zachowań bardziej elastycznych oraz adaptujących się do zmian w środowisku gry, ponieważ postać

może dzięki takiemu rozwiązaniu dynamicznie modyfikować plan w odpowiedzi na nowe informacje.

Istotną rolę w zachowaniu NPC odgrywają również algorytmy nawigacyjne oraz modele ruchu. Algorytmy wyszukiwania ścieżek takie jak A* umożliwiają wyznaczanie optymalnych tras pomiędzy punktami w środowisku gry, uwzględniając przeszkody oraz ograniczenia terenu. Modele ruchu odpowiadają natomiast za płynną realizację zaplanowanej trajektorii, kontrolę prędkości, przyspieszenia oraz interakcję z systemem kolizji. Integracja algorytmów nawigacyjnych z systemami decyzyjnymi pozwala na tworzenie postaci zdolnych do samodzielnego poruszania się po złożonych poziomach oraz reagowania na napotkane przeszkody w czasie rzeczywistym.

W ostatnich latach coraz większe zainteresowanie budzą również modele oparte na uczeniu maszynowym, w tym sieci neuronowe oraz algorytmy uczenia ze wzmocnieniem. W odróżnieniu od klasycznych podejść, w których zachowanie jest bezpośrednio zaprogramowane przez twórcę, modele uczące się potrafią modyfikować podejmowane decyzje na podstawie doświadczeń zdobytych w trakcie rozgrywki lub procesów treningowych. Takie rozwiązania pozwalają na tworzenie zachowań adaptacyjnych oraz mniej przewidywalnych, jednak wiążą się z wyzwaniami, takimi jak trudność w interpretacji decyzji modelu, zwiększony koszt testowania oraz ryzyko niepożądanych zachowań z punktu widzenia balansu gry.

Podsumowując, algorytmy i modele wspomagające zachowania NPC obejmują szeroki wachlarz rozwiązań, od prostych systemów regułowych i maszyn stanów, poprzez drzewa behawioralne i planowanie działań, po nowoczesne metody wykorzystujące uczenie maszynowe. Dobór odpowiedniego modelu zależy od charakteru tworzonej gry, roli pełnionej przez postać niezależną oraz wymagań projektowych dotyczących realizmu, kontroli i wydajności. Szczegółowe omówienie wybranych technik, wraz z ich implementacją w projekcie towarzyszącym pracy, zostało zawarte w rozdziale 3.

1.4 Personalizacja zachowań NPC z wykorzystaniem AI

Personalizacja zachowań postaci niezależnych oznacza dostosowywanie sposobu działania NPC do kontekstu rozgrywki — z uwzględnieniem zachowania gracza, stanu świata gry oraz indywidualnych cech samej postaci. Celem nie jest wyłącznie regulowanie poziomu trudności, lecz przede wszystkim zachowanie spójności świata gry oraz zwiększenie naturalności interakcji z NPC, co przekłada się na silniejsze zaangażowanie użytkownika.

Tradycyjny model projektowania zachowań NPC opiera się na stałych regułach ustalanych w fazie produkcji — zachowanie postaci pozostaje więc niezmiennie niezależnie od przebiegu rozgrywki. Rozwiązanie takie jest łatwe do wdrożenia i testowania, jednak sprzyja monotonii: powtarzalne schematy reakcji przeciwników osłabiają immersję oraz zmniejszają atrakcyjność kolejnych podejść do gry.

Odpowiedzią na te ograniczenia jest personalizacja zachowań, przyjmująca różne formy w zależności od skali projektu i dostępnych zasobów. Popularnym rozwiązaniem jest dynamiczne dostosowywanie poziomu trudności (Dynamic Difficulty Adjustment, DDA), polegające na regulowaniu zachowań przeciwników w celu utrzymania równowagi między wyzwaniem a satysfakcją gracza. Gdy wyniki gracza wskazują na zbyt wysoki poziom trudności, system może obniżyć agresywność NPC lub zmniejszyć czułość ich percepcji; w sytuacji odwrotnej — zintensyfikować ich działania. Celem takich technik jest utrzymanie płynności rozgrywki przy zachowaniu poczucia uczciwości względem gracza.

Kolejny aspekt personalizacji to modelowanie stylu gry użytkownika i dostosowywanie do niego zachowań NPC. Wielu graczy preferuje różne strategie — część z nich nastawiona jest na agresywną walkę, inni unikają konfliktów lub wybierają eksplorację. Systemy personalizacyjne mogą reagować na te preferencje, zmieniając sposób, w jaki NPC podejmują decyzje. Między innymi, gdy gracz często atakuje bezpośrednio, przeciwnicy mogą zacząć częściej unikać ciosów lub współpracować ściślej z innymi NPC, by stawić mu czoła. Jeśli natomiast ktoś woli skradanie się, NPC mogą być bardziej czujni i uwzględniać nietypowe sytuacje w patrolach — reagować na obecność gracza lub modyfikować trasy patroli. To sprawia, że świat gry wydaje się bardziej „żywy” i reagujący na działania gracza.

Personalizacja dotyczy również indywidualnych cech postaci niezależnych. Oznacza to, że NPC nie muszą stanowić powtarzalnych kopii kierowanych wspólnym schematem, lecz mogą mieć własne parametry wpływające na to, jak się zachowują. Do takich cech można zaliczyć choćby ostrożność, skłonność do współpracy, agresywność czy preferowany dystans walki. Te parametry mogą być ustalone z góry lub zmieniać się dynamicznie podczas gry, zależnie od tego, jak przebiega rozgrywka. Przykładowo, NPC może stawać się bardziej ostrożny po kilku porażkach w starciach z graczem albo łagodnieć, gdy gracz jest osłabiony. Dzięki temu nawet niewielka liczba typów przeciwników może zachowywać się różnorodnie. W projekcie praktycznym przedstawiono dwa typy przeciwników, które różnią się nie tylko parametrami, lecz także całymi automatami stanów. Parametry zachowania każdego z nich są przy tym wystawione w inspektorze silnika, dzięki czemu można stroić je niezależnie.

Ważną kwestią w personalizacji jest również pamięć i uwzględnianie historii relacji między graczem a NPC. W grach, w których postacie niezależne pełnią funkcje fabularne lub są sojusznikami, system może brać pod uwagę wcześniejsze decyzje gracza, wykonane zadania czy dotychczasowy sposób traktowania danej postaci. W efekcie zachowanie postaci może się zmieniać — na przykład rośnie zaufanie lub pojawia się wrogość. Zależnie od charakteru gry, takie mechanizmy mogą być stosowane w znacznie bardziej rozbudowany sposób albo w uproszczeniu, np. poprzez zmiany w dialogach czy reakcjach NPC.

Od strony technicznej personalizacja zachowań NPC może być realizowana zarówno przez klasyczne systemy sztucznej inteligencji, jak i przez uczenie maszynowe. W tradycyjnych rozwiązaniach personalizacja najczęściej sprowadza się do zmiany parametrów istniejących struktur decyzyjnych. Na przykład w maszynach stanów można modyfikować progi przejść między poszczególnymi stanami, a w drzewach behawioralnych aktywować lub wyłączać określone gałęzie w zależności od sytuacji. W systemach planowania celów (GOAP) personalizacja może polegać na dostosowywaniu wag albo kosztów różnych akcji, co z kolei oznacza, że NPC wybierają inne strategie.

Z drugiej strony, metody uczenia maszynowego pozwalają na tworzenie złożonych modeli, które uczą się na podstawie zebranych danych rozgrywki. Przykładowo, można trenować algorytm przewidujący zachowania gracza lub oceniający jego umiejętności i na tej podstawie zmieniać parametry NPC. Niemniej jednak w praktyce jest to dość trudne do

wdrożenia w typowych grach, głównie ze względu na koszty trenowania modeli, trudności z interpretacją ich decyzji oraz konieczność zachowania stabilności działania w czasie rzeczywistym. Z tego powodu wielu twórców korzysta z rozwiązań mieszanych, w których uczenie maszynowe służy głównie do analizy danych i formułowania rekomendacji, podczas gdy ostateczne decyzje pozostają pod kontrolą reguł lub innych struktur decyzyjnych.

Personalizacja zachowań NPC niesie ze sobą oczywiście wiele korzyści, ale także pewne zagrożenia na etapie projektowania. Do najczęstszych problemów należy utrata spójności zachowań. Jeśli system personalizacji działa zbyt intensywnie lub nie jest odpowiednio ograniczony, NPC mogą zacząć zachowywać się w sposób trudny do przewidzenia, a nawet chaotyczny. Z perspektywy gracza takie zachowania mogą wydawać się losowe, niesprawiedliwe albo wręcz niezgodne z ustaloną logiką świata gry. Ponadto istnieje ryzyko zaburzenia balansu rozgrywki, zwłaszcza gdy NPC zbyt dobrze przystosowują się do strategii gracza, co może podnieść trudność ponad miarę i doprowadzić do frustracji i zniechęcenia odbiorcy.

Podsumowując, personalizacja zachowań NPC to ważny element nowoczesnego projektowania gier, pozwalający na tworzenie bardziej żywych i angażujących doświadczeń. Może ona przybierać różne formy — od dostosowywania poziomu trudności, przez reagowanie na styl gry użytkownika, aż po modelowanie indywidualnych cech postaci czy uwzględnianie historii interakcji. W zależności od zastosowanych rozwiązań technicznych, personalizacja bywa realizowana zarówno przez klasyczne metody, jak i nowocześniejsze podejścia oparte na uczeniu maszynowym. Zawsze jednak pozostaje wyzwaniem utrzymanie kontroli nad spójnością zachowań oraz równowagą rozgrywki, tak aby odbiorca nie odczuwał personalizacji jako sztucznej ingerencji, lecz jako coś naturalnego i wzbogacającego jego doświadczenie.

1.5 Etyczne i prawne aspekty wykorzystania AI w grach

Wykorzystanie sztucznej inteligencji w grach komputerowych, w tym przy projektowaniu zachowań postaci niezależnych, wiąże się nie tylko z możliwościami technologicznymi, ale również z odpowiedzialnością etyczną i prawną. Systemy AI wpływają na sposób, w jaki gracz doświadcza świata gry, jak interpretuje zachowania NPC oraz jak reaguje na decyzje podejmowane przez algorytmy. W przypadku prostych systemów opartych na regułach ryzyko etyczne może wydawać się ograniczone, jednak wraz ze wzrostem złożoności algorytmów, personalizacji oraz analizy danych użytkownika pojawia się potrzeba świadomego projektowania takich rozwiązań.

Jednym z podstawowych problemów etycznych jest przejrzystość działania systemów sztucznej inteligencji. Gracz nie musi znać szczegółów implementacji algorytmu, jednak powinien mieć poczucie, że zachowania NPC wynikają z zasad świata gry, a nie z losowych lub nieuczciwych decyzji systemu. Jeżeli przeciwnik reaguje na działania gracza w sposób, który wydaje się niemożliwy do przewidzenia lub sprzeczny z zasadami rozgrywki, może to prowadzić do frustracji i utraty zaufania do reguł gry. Z tego względu projektowanie AI powinno uwzględniać nie tylko skuteczność zachowania NPC, ale także jego czytelność i zgodność z oczekiwaniami użytkownika.

Szczególne znaczenie ma problem tak zwanej „uczciwości” sztucznej inteligencji w grach. NPC nie powinien sprawiać wrażenia, że posiada wiedzę, której nie mógł zdobyć w ramach logiki świata gry. Przykładem niepożądanego rozwiązania byłaby sytuacja, w której przeciwnik bez żadnego kontaktu wzrokowego lub dźwiękowego natychmiast zna dokładną pozycję gracza. Aby zapobiegać takim sytuacjom, stosuje się ograniczenia percepcji NPC, o których była mowa w podrozdziale 1.1.

Kolejnym zagadnieniem etycznym jest nadmierna manipulacja doświadczeniem gracza. Rozwiązania adaptacyjne, opisane w podrozdziale 1.4, mogą poprawiać komfort gry, jednak ich zastosowanie powinno być wyważone. Jeżeli gracz zorientuje się, że gra zbyt mocno dostosowuje się do jego wyników, może odczuć, że sukcesy nie wynikają z jego umiejętności, lecz z ukrytej ingerencji systemu. Z drugiej strony zbyt agresywne zwiększanie trudności może prowadzić do frustracji. Etyczne projektowanie adaptacyjnej sztucznej inteligencji

wymaga więc znalezienia równowagi pomiędzy wsparciem gracza a zachowaniem autentycznego poczucia wyzwania.

Istotnym aspektem jest również odpowiedzialność projektanta i programisty za działanie systemu AI. Nawet jeżeli zachowanie NPC jest generowane przez złożony algorytm lub model uczący się, odpowiedzialność za jego wdrożenie pozostaje po stronie twórców gry. Oznacza to konieczność testowania zachowań postaci niezależnych, przewidywania możliwych błędów oraz ograniczania sytuacji, w których AI mogłaby zachowywać się w sposób niepożądany. Dotyczy to szczególnie systemów personalizacji i uczenia maszynowego, w których decyzje algorytmu mogą być trudniejsze do zinterpretowania niż w przypadku klasycznych maszyn stanów lub drzew behawioralnych.

W kontekście prawnym duże znaczenie ma ochrona danych użytkownika. Jeżeli gra analizuje styl gry, czas reakcji, częstotliwość porażek, podejmowane decyzje lub inne informacje związane z zachowaniem gracza, dane te mogą stać się podstawą do personalizacji rozgrywki. W prostym prototypie dane takie mogą być wykorzystywane lokalnie i tymczasowo, bez tworzenia profilu użytkownika. W bardziej rozbudowanych produkcjach, zwłaszcza sieciowych, analiza zachowań gracza może jednak wiązać się z przetwarzaniem danych osobowych, w tym danych o zachowaniu użytkownika. W takiej sytuacji konieczne jest przestrzeganie zasad przejrzystości, minimalizacji danych oraz informowania użytkownika o sposobie ich wykorzystania. W części praktycznej projektu żadne dane użytkownika nie są zapisywane – cała logika gry działa na bieżących stanach w pamięci.

Z punktu widzenia RODO szczególnie ważne jest, aby twórcy systemu nie gromadzili danych w zakresie większym, niż jest to rzeczywiście potrzebne do działania gry. Jeżeli personalizacja NPC wymaga jedynie lokalnego zapamiętania liczby porażek gracza lub jego ostatniego stylu działania, nie ma uzasadnienia dla zbierania dodatkowych informacji identyfikujących użytkownika. Minimalizacja danych jest ważna nie tylko z punktu widzenia prawa, ale również z perspektywy bezpieczeństwa projektu. Im mniej wrażliwych informacji

przechowuje system, tym mniejsze ryzyko nadużyć, wycieku danych lub nieuprawnionego profilowania gracza.²⁶

Kolejnym problemem jest automatyczne podejmowanie decyzji i profilowanie użytkownika. W grach komputerowych profilowanie może przyjmować formę analizy sposobu gry, preferencji, skuteczności, stylu walki lub wyborów fabularnych. Samo dostosowywanie rozgrywki do zachowania użytkownika nie musi być problematyczne, o ile odbywa się w sposób przejrzysty i nie prowadzi do istotnych negatywnych konsekwencji dla gracza. Ryzyko pojawia się wtedy, gdy dane są wykorzystywane szerzej, na przykład do manipulowania decyzjami użytkownika, projektowania nadmiernie angażujących mechanizmów lub wpływania na zachowania zakupowe w grach z mikropłatnościami.

W przypadku gier wykorzystujących sztuczną inteligencję należy również zwrócić uwagę na kwestie związane z uzależniającym projektowaniem mechanik. AI może zostać wykorzystana do analizowania momentów, w których gracz jest najbardziej podatny na kontynuowanie rozgrywki, zakup dodatków lub podejmowanie określonych działań. Chociaż personalizacja może poprawiać komfort użytkownika, może też zostać użyta w sposób nieetyczny, na przykład do nadmiernego wydłużania czasu gry lub zwiększania presji na zakup elementów dodatkowych. Z tego powodu odpowiedzialne projektowanie AI powinno uwzględniać dobro użytkownika, a nie wyłącznie maksymalizację zaangażowania.

Osobnym zagadnieniem są prawa autorskie i własność intelektualna. Współczesne narzędzia AI mogą wspierać tworzenie grafik, animacji, dialogów, muzyki, kodu lub koncepcji zachowań NPC. Jeżeli twórca gry korzysta z takich narzędzi, powinien zweryfikować, czy wygenerowane materiały mogą być legalnie wykorzystane w projekcie. Problem ten jest szczególnie istotny w przypadku generatywnej sztucznej inteligencji, ponieważ nie zawsze jest jasne, na jakich danych trenowany był model oraz czy wygenerowany rezultat nie narusza praw osób trzecich. W pracy inżynierskiej ma to znaczenie zwłaszcza wtedy, gdy projekt gry zawiera zasoby graficzne, muzyczne lub kod wygenerowany przy pomocy narzędzi AI. W grze

²⁶ Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych (RODO), Dz. Urz. UE L 119 z 4.05.2016.

stworzonej na potrzeby pracy wykorzystano gotowe pakiety assetów, po uprzedniej weryfikacji warunków ich licencji.²⁷

W kontekście programowania NPC istotne jest również rozróżnienie pomiędzy wykorzystaniem AI jako elementu działania gry a wykorzystaniem AI jako narzędzia wspomagającego proces tworzenia. Jeżeli sztuczna inteligencja steruje zachowaniem NPC w gotowej grze, należy analizować jej wpływ na rozgrywkę, balans i dane użytkownika. Jeżeli natomiast AI jest używana jako narzędzie pomocnicze podczas tworzenia kodu, grafik lub dokumentacji, ważne stają się kwestie praw autorskich, poprawności wygenerowanych materiałów oraz odpowiedzialności autora projektu za ich weryfikację. W obu przypadkach AI nie zwalnia twórcy z obowiązku kontroli jakości i zgodności rozwiązania z przyjętymi założeniami.

Warto również odnieść się do regulacji dotyczących sztucznej inteligencji, takich jak rozporządzenie Parlamentu Europejskiego i Rady (UE) 2024/1689 w sprawie sztucznej inteligencji (tzw. AI Act). Jego celem jest określenie zasad bezpiecznego i odpowiedzialnego stosowania systemów AI, szczególnie tam, gdzie mogą one wpływać na prawa, bezpieczeństwo lub sytuację użytkowników. Większość prostych systemów AI stosowanych w grach komputerowych, takich jak maszyny stanów sterujące przeciwnikiem, nie będzie należeć do najbardziej ryzykownych kategorii. Nie oznacza to jednak, że twórcy gier mogą pomijać zasady przejrzystości, bezpieczeństwa i odpowiedzialności. Nawet w niewielkim projekcie warto projektować systemy w taki sposób, aby ich działanie było możliwe do wyjaśnienia, przetestowania i ograniczenia w razie błędów.²⁸

Etyczne projektowanie AI w grach wymaga również dbałości o brak dyskryminacji i niepożądanych uprzedzeń. W przypadku prostych przeciwników platformowych problem ten może wydawać się odległy, jednak w większych grach NPC mogą reagować na decyzje gracza, jego styl komunikacji, wybory fabularne albo sposób interakcji z innymi postaciami. Jeżeli algorytmy uczące się są trenowane na nieodpowiednich danych, mogą utrwalać niepożądane wzorce lub prowadzić do nierównego traktowania użytkowników. Z tego

²⁷ Ustawa z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych, Dz.U. 1994 nr 24 poz. 83 ze zm.

²⁸ Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2024/1689 z dnia 13 czerwca 2024 r. ustanawiające zharmonizowane przepisy dotyczące sztucznej inteligencji, Dz. Urz. UE L 2024/1689.

względu twórcy powinni zachować ostrożność przy wykorzystywaniu danych treningowych i testować system pod kątem niezamierzonych skutków.

Ważnym elementem odpowiedzialnego wdrażania AI jest także możliwość testowania i debugowania zachowań NPC. System, którego działania nie da się prześledzić, jest trudny do kontrolowania, a tym samym mniej bezpieczny projektowo. Z tego powodu w wielu przypadkach klasyczne techniki, takie jak maszyny stanów, drzewa behawioralne czy systemy regułowe, są korzystne nie tylko ze względu na prostotę implementacji, ale również ze względu na przejrzystość. Programista może łatwo ustalić, dlaczego NPC przeszedł do konkretnego stanu, rozpoczął atak albo zakończył pościg. W przypadku modeli uczących się taka interpretacja bywa znacznie trudniejsza.

Podsumowując, etyczne i prawne aspekty wykorzystania AI w grach komputerowych obejmują wiele zagadnień: przejrzystość działania algorytmów, uczciwość zachowań NPC, ochronę danych gracza, odpowiedzialność twórców, prawa autorskie, bezpieczeństwo oraz ryzyko nadmiernej manipulacji użytkownikiem. W przypadku projektowania postaci niezależnych szczególne znaczenie ma zachowanie równowagi pomiędzy skutecznością sztucznej inteligencji a czytelnością i sprawiedliwością rozgrywki. Odpowiedzialne wdrażanie AI nie polega więc wyłącznie na tworzeniu coraz bardziej złożonych systemów, lecz na takim ich projektowaniu, aby wspierały jakość gry, nie naruszały praw użytkownika i pozostawały pod kontrolą twórców.

2 Praktyczne wykorzystanie AI w procesie programowania NPC

Niniejszy rozdział przedstawia praktyczną część pracy, obejmującą projekt, implementację oraz analizę gry platformowej 2D wykorzystującej techniki sztucznej inteligencji do sterowania zachowaniem postaci niezależnych. Projekt został zrealizowany w silniku Unity z wykorzystaniem języka C#, a jego głównym celem było opracowanie dwóch typów przeciwników o czytelnych i przewidywalnych zachowaniach, zrealizowanych w oparciu o skończone automaty stanów (Finite State Machine).

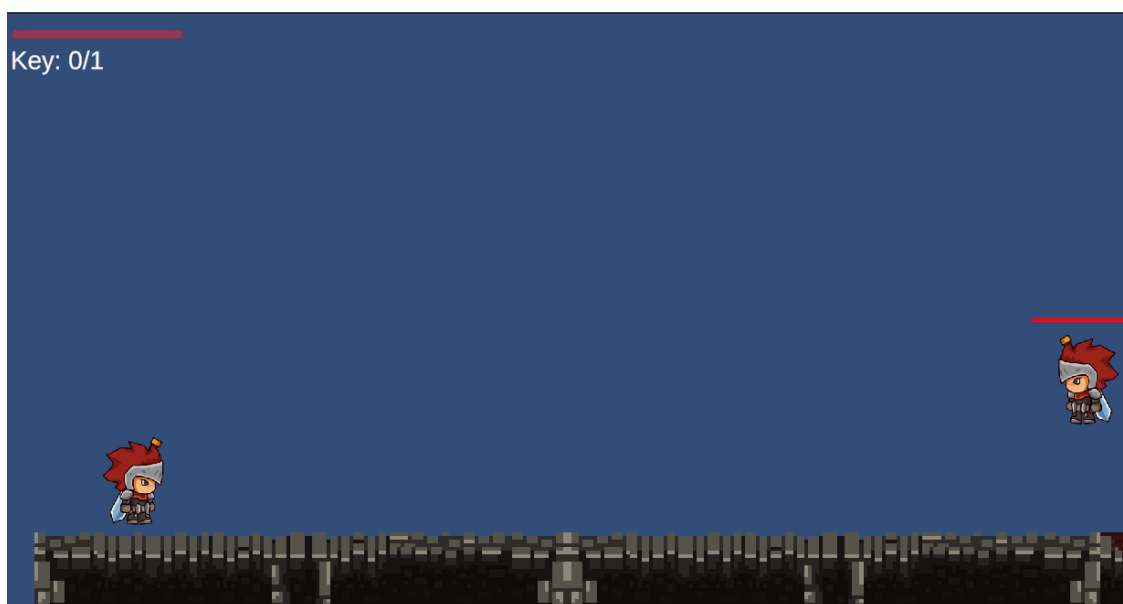
W rozdziale przedstawiono kolejne etapy realizacji projektu, począwszy od omówienia założeń projektowych oraz roli przeciwników w grze, poprzez uzasadnienie doboru zastosowanych narzędzi i technologii, aż do opisu procesu implementacji poszczególnych mechanizmów sztucznej inteligencji. Następnie zaprezentowano sposób prowadzenia testów opracowanego rozwiązania oraz wnioski wynikające z jego realizacji i działania.

2.1 Założenia projektowe i rola NPC w grze

Celem części praktycznej pracy było opracowanie prototypu dwuwymiarowej gry platformowej, w której zachowaniem postaci niezależnych sterują systemy sztucznej inteligencji. Zamiast budowy rozbudowanej gry komercyjnej skoncentrowano się na implementacji mechanizmów odpowiedzialnych za zachowanie przeciwników oraz interakcję pomiędzy graczem a sterowanymi przez sztuczną inteligencję postaciami. Rozgrywka została zaprojektowana w klasycznej konwencji platformówki 2D.

Gracz porusza się po poziomie, pokonując przeszkody terenowe oraz przeciwników, a jego zadaniem jest odnalezienie klucza odblokowującego możliwość ukończenia poziomu, a następnie dotarcie do flagi oznaczającej zakończenie gry. Taki schemat rozgrywki zapewnia eksplorację całej planszy oraz kontakt z postaciami niezależnymi rozmieszczonymi w różnych jej częściach.

Sterowana przez użytkownika postać została wyposażona w podstawowe mechaniki charakterystyczne dla gier platformowych. Gracz może poruszać się w obu kierunkach, wykonywać skoki, zadawać obrażenia przeciwnikom za pomocą ataku wręcz oraz otrzymywać obrażenia podczas kontaktu z wrogami. W projekcie zaimplementowano również system punktów życia, którego wyczerpanie powoduje zakończenie rozgrywki i wyświetlenie ekranu *Game Over*.



Rysunek 2.1. Widok rozgrywki z perspektywy gracza, przedstawiający postać sterowaną przez użytkownika oraz elementy interfejsu (pasek życia, licznik kluczy).

Źródło: Opracowanie własne.

Kluczowym elementem projektu są postacie niezależne, których zadaniem nie jest jedynie blokowanie przejścia, lecz aktywne reagowanie na działania gracza. W projekcie zastosowano dwa typy przeciwników pełniących odmienne funkcje. Pierwszy z nich patroluje wyznaczony fragment planszy i rozpoczyna pościg po wykryciu gracza znajdującego się w określonym zasięgu. Drugi pełni rolę strażnika przypisanego do konkretnego obszaru mapy. Nie przemieszcza się pomiędzy punktami patrolowymi, lecz obserwuje otoczenie, reagując dopiero po wykryciu gracza znajdującego się w pobliżu.

Podczas projektowania zachowań NPC przyjęto założenie, że przeciwnicy powinni stanowić wyzwanie dla gracza, jednocześnie pozostając przewidywalni i zrozumiali. Z tego względu zrezygnowano z rozwiązań opartych na losowości oraz mechanizmów mogących sprawiać wrażenie nieuczciwych. Każda reakcja przeciwnika wynika z wcześniej zdefiniowanych warunków, takich jak odległość od gracza, wykryte krawędzie platformy lub osiągnięcie granicy patrolowanego obszaru. Dzięki temu użytkownik jest w stanie obserwować zachowanie przeciwników i przewidywać ich kolejne działania, co pozytywnie wpływa na

odbiór rozgrywki.

Świadomie ograniczono również zakres projektu. Prototyp obejmuje pojedynczy poziom oraz dwa typy przeciwników, a do sterowania ich zachowaniem wykorzystano proste, przejrzyste automaty stanów zamiast bardziej zaawansowanych technik opisanych w rozdziale pierwszym. Ograniczenia te nie wynikały z braków technicznych, lecz z przyjętego celu pracy — skupienia się na czytelnej implementacji i analizie zachowań NPC. Uzasadnienie doboru zastosowanych rozwiązań przedstawiono w podrozdziale 2.2.

Przyjęte założenia projektowe są zgodne z wnioskami przedstawionymi w pierwszym rozdziale pracy, w myśl których skuteczność sztucznej inteligencji w grach komputerowych nie wynika z maksymalizacji poziomu trudności, lecz z tworzenia zachowań spójnych, czytelnych oraz odpowiadających zasadom obowiązującym w świecie gry.

2.2 Dobór narzędzi AI do tworzenia zachowań NPC

Po określeniu założeń funkcjonalnych projektu kolejnym etapem było wybranie technologii umożliwiających implementację systemu sztucznej inteligencji oraz pozostałych mechanizmów gry. Podczas podejmowania decyzji kierowano się przede wszystkim

możliwością szybkiego prototypowania, dostępnością narzędzi wspierających tworzenie gier dwuwymiarowych oraz łatwością integracji poszczególnych komponentów odpowiedzialnych za logikę rozgrywki.

Projekt został zrealizowany z wykorzystaniem silnika Unity, który obecnie należy do popularniejszych środowisk do tworzenia gier komputerowych, zarówno w zastosowaniach komercyjnych, jak i edukacyjnych. Wybór tego środowiska był podyktowany przede wszystkim rozbudowanym systemem komponentów, wsparciem dla fizyki 2D oraz możliwością łatwego łączenia logiki programu z elementami wizualnymi. Istotnym argumentem była również możliwość testowania zmian w czasie rzeczywistym, co znacząco usprawniło proces implementacji oraz późniejszego strojenia zachowań postaci.²⁹

Warstwa programistyczna projektu została zaimplementowana w języku C#, który stanowi podstawowy język skryptowy wykorzystywany przez silnik Unity. Dzięki programowaniu obiektowemu możliwe było rozdzielenie odpowiedzialności pomiędzy poszczególne komponenty odpowiadające za ruch postaci, obsługę punktów życia, system walki oraz logikę działania sztucznej inteligencji. Takie podejście zwiększyło przejrzystość projektu oraz ułatwiło jego dalszą rozbudowę.

Jednym z najważniejszych etapów projektowania było wybranie modelu odpowiedzialnego za sterowanie zachowaniem przeciwników. Po przeanalizowaniu rozwiązań przedstawionych w części teoretycznej zdecydowano się na wykorzystanie skończonych automatów stanów (Finite State Machine). Wybór ten wynikał przede wszystkim z charakteru opracowywanej gry. W projekcie występują dwa typy przeciwników realizujące stosunkowo niewielką liczbę jasno określonych zachowań, dlatego zastosowanie bardziej złożonych modeli, takich jak drzewa behawioralne czy systemy planowania celów, byłoby nieuzasadnione i prowadziłoby przede wszystkim do zwiększenia złożoności implementacji.

Dodatkowym argumentem przemawiającym za wykorzystaniem automatów stanów była łatwość testowania oraz możliwość jednoznacznego określenia warunków przejścia pomiędzy poszczególnymi stanami. Takie rozwiązanie umożliwiło szybkie wykrywanie

²⁹ Unity Technologies, *Unity Manual — Physics 2D*, <https://docs.unity3d.com/6000.5/Documentation/Manual/2d-physics/2d-physics.html>, [dostęp: 12.08.2026].

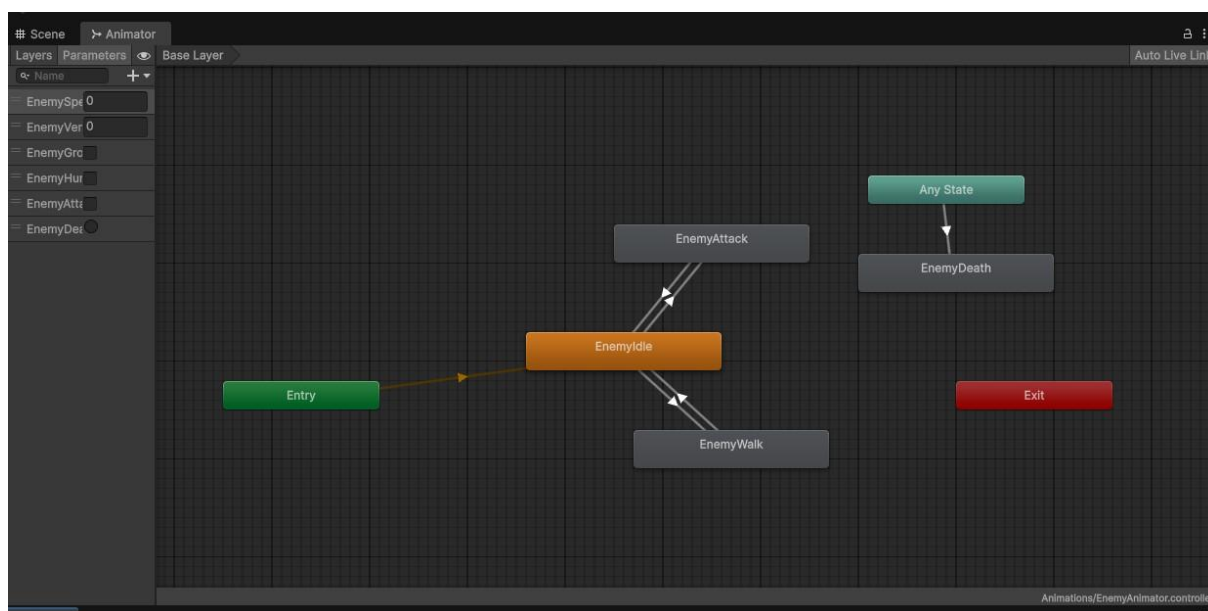
błędów podczas implementacji oraz stopniowe rozwijanie zachowań przeciwników bez konieczności przebudowy całego systemu decyzyjnego.

W projekcie świadomie zrezygnowano z zastosowania technik uczenia maszynowego. Rozwiązania tego typu wymagają przygotowania zbiorów danych treningowych oraz znacznie bardziej złożonego procesu weryfikacji poprawności działania modeli. W przypadku niewielkiej gry platformowej wykorzystanie uczenia maszynowego nie przyniosłoby istotnych korzyści, jednocześnie znacząco komplikując proces implementacji oraz późniejszego testowania.

Podobną decyzję podjęto w odniesieniu do systemów automatycznej nawigacji wykorzystujących siatki nawigacyjne (NavMesh). Ze względu na dwuwymiarowy charakter projektu oraz stosunkowo prostą konstrukcję planszy ruch przeciwników został oparty na bezpośrednim sterowaniu oraz mechanizmach wykrywania przeszkód i krawędzi platform. Takie rozwiązanie okazało się wystarczające do realizacji wszystkich założeń projektowych, przy jednoczesnym ograniczeniu kosztów obliczeniowych oraz złożoności implementacji.

Uzupełnieniem warstwy programistycznej był system Animator Controller wykorzystywany do synchronizacji logiki programu z animacjami postaci. Pozwolił on na oddzielenie mechanizmów odpowiedzialnych za decyzje podejmowane przez sztuczną inteligencję od warstwy wizualnej, dzięki czemu możliwe było niezależne rozwijanie obu elementów projektu.³⁰

³⁰ Unity Technologies, *Unity Manual* — *Animator Controller*, dz. cyt.



Rysunek 2.2. Automat stanów animacji przeciwnika (Animator Controller) w edytorze Unity.

Źródło: opracowanie własne

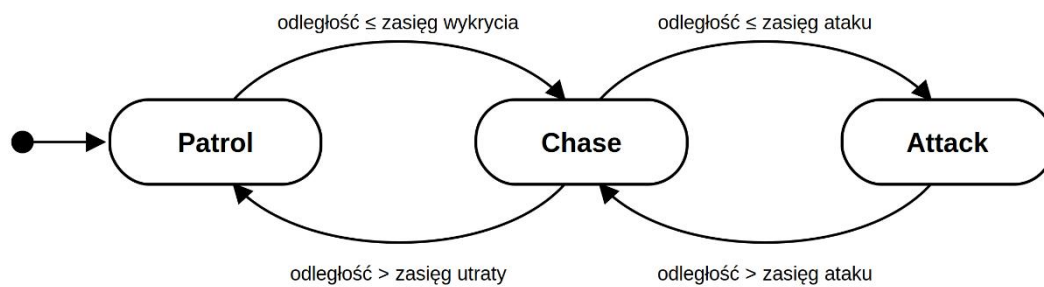
2.3 Proces projektowania i implementacji NPC wspomaganych AI

Najważniejszym elementem części praktycznej było opracowanie systemu sztucznej inteligencji odpowiedzialnego za sterowanie zachowaniem przeciwników. Podczas projektowania przyjęto założenie, że logika NPC powinna zostać podzielona na niezależne komponenty odpowiadające za określone funkcje. Pozwoliło to zachować przejrzystość kodu oraz ograniczyć zależności pomiędzy poszczególnymi elementami systemu.

Każdy przeciwnik został zbudowany jako zestaw współpracujących komponentów realizujących odrębne zadania. Głównym elementem odpowiedzialnym za podejmowanie decyzji jest automat stanów sterujący przebiegiem zachowania NPC. Niezależnie od niego funkcjonują komponenty odpowiedzialne za obsługę punktów życia, wyświetlanie informacji o stanie zdrowia, odtwarzanie animacji oraz reakcję na otrzymywane obrażenia. Taki podział umożliwił ponowne wykorzystanie części kodu podczas implementacji drugiego rodzaju przeciwnika oraz ułatwił późniejsze testowanie poszczególnych mechanizmów.

Pierwszym zaimplementowanym typem przeciwnika był przeciwnik patrolujący wyznaczony

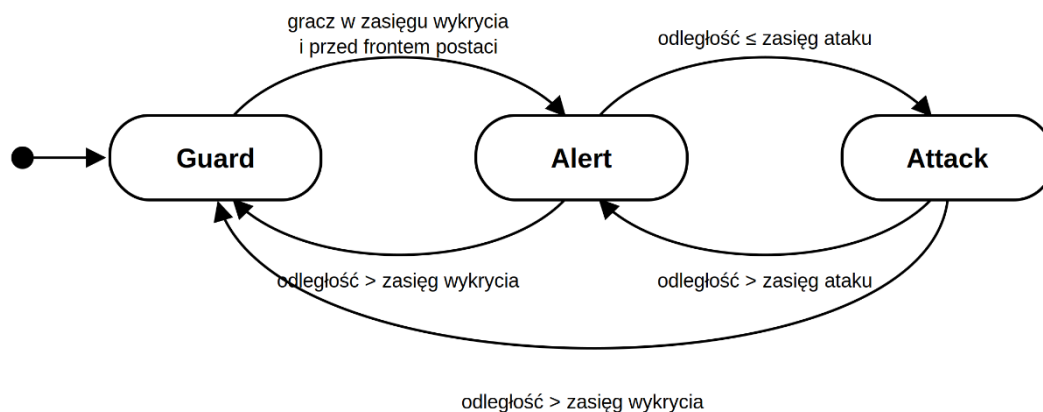
fragment planszy. Jego zachowanie zostało oparte na automacie stanów składającym się z trzech podstawowych stanów: patrolowania, pościgu za graczem oraz ataku (rys. 2.1). Po utracie kontaktu z graczem przeciwnik powraca do patrolowania poprzez stan pościgu. Przejścia pomiędzy poszczególnymi stanami realizowane są na podstawie informacji pozyskiwanych z systemu percepcji oraz aktualnej odległości od gracza.



Rysunek 2.3. Diagram stanów przeciwnika patrołującego.

Źródło: opracowanie własne.

Drugim typem przeciwnika jest strażnik odpowiedzialny za ochronę określonego fragmentu planszy. W przeciwieństwie do przeciwnika patrołującego nie porusza się on pomiędzy punktami patrołowymi, lecz pozostaje w jednym miejscu do momentu wykrycia gracza. Po wejściu użytkownika w obszar przechodzi w stan alarmowy, w którym obraca się w kierunku gracza, a następnie przechodzi w stan ataku (rys. 2.2). Strażnik w odróżnieniu od przeciwnika patrołującego posiada bezpośrednie przejście ze stanu ataku do stanu spoczynku. Takie rozwiązanie pozwoliło uzyskać dwa wyraźnie różniące się style zachowania przy zachowaniu wspólnej architektury systemu AI.



Rysunek 2.4. Diagram stanów przeciwnika typu strażnik.

Źródło: opracowanie własne.

Istotnym elementem implementacji był również system percepcji przeciwników. Każdy NPC wykorzystuje zdefiniowany zasięg wykrywania gracza, natomiast opuszczenie tego obszaru nie powoduje natychmiastowego zakończenia pościgu. W projekcie zastosowano dwa niezależne progi odległości – pierwszy odpowiada za rozpoczęcie wykrywania, drugi za zakończenie pościgu. Takie rozwiązanie eliminuje częste przetaczanie się pomiędzy stanami w sytuacji, gdy gracz porusza się w pobliżu granicy zasięgu wykrywania, dzięki czemu zachowanie przeciwników jest bardziej stabilne i naturalne.

Ruch przeciwników został dodatkowo uzupełniony o mechanizm wykrywania krawędzi platform oraz przeszkód znajdujących się przed postacią. Pozwoliło to uniknąć sytuacji, w których przeciwnik opuszczałby wyznaczony obszar lub spadał z platformy podczas patrolowania. Zastosowanie prostego wykrywania krawędzi za pomocą raycastu w dół okazało się w pełni wystarczające dla charakteru opracowywanej gry.³¹

³¹ Unity Technologies, Unity Scripting API — Physics2D.Raycast, <https://docs.unity3d.com/6000.5/Documentation/ScriptReference/Physics2D.Raycast.html>, [dostęp: 13.08.2026].

Mechanizm zadawania obrażeń został zsynchronizowany z animacją ataku poprzez wykorzystanie zdarzeń animacyjnych (Animation Events). Dzięki temu obrażenia są zadawane dokładnie w momencie kontaktu ciosu z postacią gracza, a nie w chwili rozpoczęcia animacji. Rozwiązanie to poprawiło czytelność rozgrywki oraz zwiększyło zgodność pomiędzy warstwą wizualną a logiką programu.³²

Poza samym podejmowaniem decyzji istotnym elementem implementacji było odpowiednie reagowanie przeciwników na otrzymywane obrażenia. Po trafieniu postać zostaje na krótki czas ogłuszona, co powoduje chwilowe wstrzymanie działania automatu stanów oraz zatrzymanie wykonywanych czynności. Jednocześnie zastosowano efekt odrzucenia, dzięki któremu przeciwnik zostaje odepchnięty w kierunku przeciwnym do źródła ataku. W celu wyeliminowania możliwości wielokrotnego naliczania obrażeń podczas jednego uderzenia wprowadzono również krótkotrwały okres niewrażliwości na kolejne trafienia. Dodatkowym elementem jest krótkotrwała zmiana koloru postaci na czerwony, stanowiąca wizualne potwierdzenie otrzymania obrażeń. Zastosowanie tych mechanizmów sprawia, że gracz natychmiast otrzymuje czytelną informację zwrotną o skuteczności wykonanego ataku, co wpisuje się w przyjęte wcześniej założenie tworzenia przewidywalnych i zrozumiałych zachowań NPC.

Końcowym etapem cyklu działania przeciwnika jest obsługa jego śmierci. Po wyczerpaniu punktów życia dalsze podejmowanie decyzji zostaje całkowicie zatrzymane, a przeciwnik przestaje reagować na zdarzenia zachodzące w świecie gry. Równocześnie wyłączone zostają elementy odpowiedzialne za fizykę oraz kolizje, dzięki czemu gracz może swobodnie przejść przez ciało pokonanego przeciwnika. Następnie odtwarzana jest animacja śmierci, po której zakończeniu obiekt zostaje usunięty ze sceny. Warto podkreślić, że cała logika związana z obsługą obrażeń i śmierci została umieszczona w komponencie odpowiedzialnym za zarządzanie punktami życia, a nie w automacie stanów. Dzięki temu oba typy przeciwników korzystają z identycznego mechanizmu obsługi obrażeń pomimo różnic w sposobie

³² Unity Technologies, Unity Manual – Animation Events, <https://docs.unity3d.com/550/Documentation/Manual/animator-AnimationEvents.html> [dostęp: 13.08.2026]

podejmowania decyzji, co stanowi praktyczny przykład zastosowania architektury komponentowej opisanej na początku podrozdziału.

Przedstawione rozwiązania tworzą kompletny cykl funkcjonowania postaci niezależnej w opracowanym projekcie – od wykrywania gracza i analizy otoczenia, poprzez podejmowanie decyzji oraz wykonywanie odpowiednich działań, aż po reakcję na otrzymywane obrażenia i zakończenie działania po utracie wszystkich punktów życia. Tak zaprojektowany system pozwolił uzyskać zachowania spójne, przewidywalne oraz łatwe do dalszej rozbudowy o kolejne funkcje. Szczegółową analizę zastosowanych rozwiązań oraz ich odniesienie do zagadnień przedstawionych w części teoretycznej pracy zaprezentowano w rozdziale trzecim.

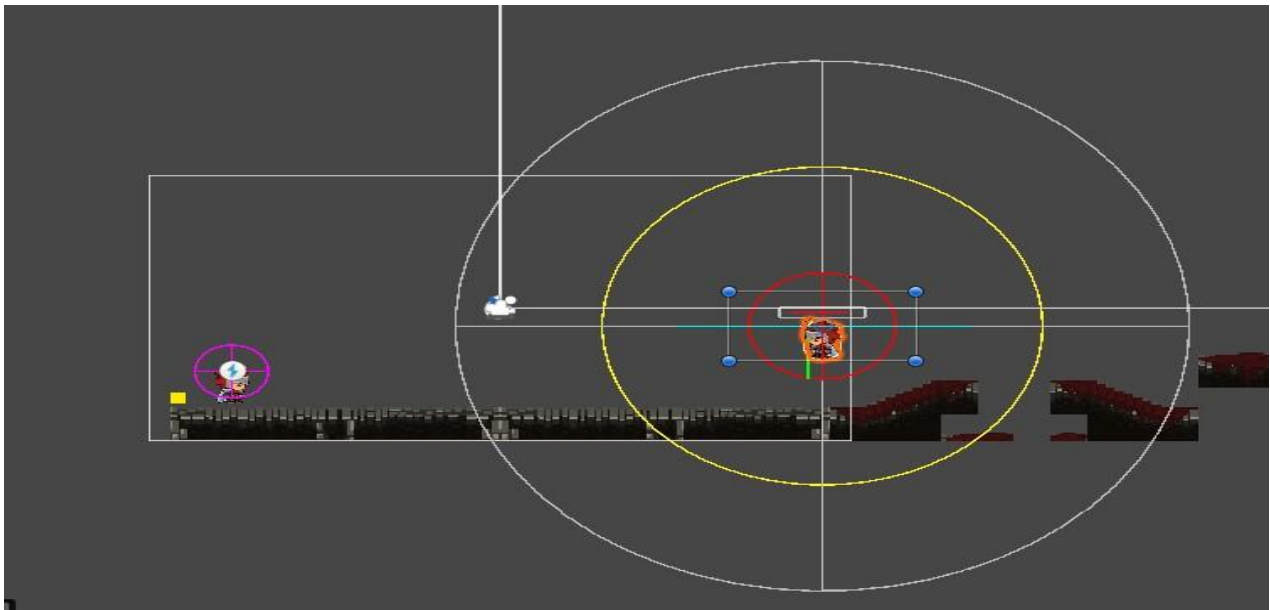
2.4 Testowanie i optymalizacja zachowań NPC

Po zakończeniu implementacji poszczególnych mechanizmów sztucznej inteligencji przeprowadzono serię testów mających na celu ocenę poprawności działania przeciwników oraz stabilności całego systemu. Proces testowania realizowano równolegle z implementacją kolejnych funkcjonalności, dzięki czemu możliwe było bieżące wykrywanie błędów oraz ich eliminacja jeszcze na etapie tworzenia projektu.

Podstawowym celem testów było sprawdzenie poprawności działania automatu stanów odpowiedzialnego za sterowanie zachowaniem przeciwników. Weryfikowano przede wszystkim poprawność przechodzenia pomiędzy stanami patrolowania, pościgu oraz ataku. Szczególną uwagę zwrócono na sytuacje graniczne, w których gracz wielokrotnie przekraczał granicę obszaru wykrywania przeciwnika. Testy sytuacji granicznych ujawniły istotny problem w pierwotnej implementacji. Wykrywanie gracza oraz zakończenie pościgu opierały się początkowo na jednym progu odległości, przez co przeciwnik gwałtownie przełączał się pomiędzy stanem patrolowania a pościgiem, gdy gracz poruszał się w pobliżu granicy zasięgu. Zachowanie takie było nieczytelne dla gracza i niestabilne. Rozwiązaniem okazało się rozdzielenie warunków na dwa niezależne progi: mniejszy, odpowiadający za rozpoczęcie pościgu, oraz większy, decydujący o jego zakończeniu. Dzięki temu przeciwnik podejmuje pościg dopiero po wyraźnym zbliżeniu się gracza, a rezygnuje z niego dopiero po jego znacznym oddaleniu, co wyeliminowało wielokrotne przełączanie stanów.

Istotnym elementem procesu testowania była również kontrola działania systemu percepcji przeciwników. W komponentach przeciwników zaimplementowano wizualizację pomocniczą (Gizmos), rysującą w edytorze zasięgi wykrywania i ataku, granice patrolu oraz promień testu podłoża. Pozwoliło to na szybkie dostrajanie parametrów odpowiedzialnych za wykrywanie gracza oraz ocenę wpływu zmian na przebieg rozgrywki. Dzięki temu możliwe było dobranie takich wartości, które zapewniały odpowiedni poziom trudności przy jednoczesnym zachowaniu przewidywalności zachowań przeciwników.³³

³³ Unity Technologies, *Unity Manual — Gizmos*, <https://docs.unity3d.com/6000.2/Documentation/Manual/gizmos-and-handles.html>, [dostęp: 15.08.2026].



Rysunek 2.5. Wizualizacja zasięgów wykrywania oraz granic patrolu w edytorze Unity (Gizmos).

Źródło: opracowanie własne

Osobnym elementem testów była weryfikacja zachowania przeciwnika typu strażnik, którego percepcja opiera się nie tylko na odległości, lecz również na kierunku zwrotu postaci. Sprawdzono przede wszystkim, czy strażnik reaguje wyłącznie na gracza znajdującego się przed nim, ignorując zbliżenie się od tyłu do momentu, w którym cykliczne obracanie się nie skieruje go w stronę gracza. Testy potwierdziły również poprawność powrotu do stanu spoczynku po oddaleniu się gracza, w tym bezpośrednio ze stanu ataku.

Podczas implementacji pojawiły się również problemy związane z synchronizacją logiki programu z animacjami postaci. W początkowych wersjach projektu możliwe było wielokrotne uruchamianie animacji ataku jeszcze przed zakończeniem poprzedniej sekwencji, co prowadziło do nieprawidłowego działania przeciwników oraz zadawania obrażeń w niezamierzonych momentach. Problem rozwiązano poprzez wprowadzenie dodatkowych warunków kontrolujących możliwość rozpoczęcia kolejnej animacji oraz synchronizację momentu zadawania obrażeń z wykorzystaniem zdarzeń Animation Event.

Kolejnym zagadnieniem wymagającym optymalizacji było zachowanie przeciwników po zakończeniu wykonywania ataku. W niektórych sytuacjach postać pozostawała w niewłaściwym stanie animacji pomimo zmiany aktualnego stanu automatu. Rozwiązaniem

okazało się wymuszenie powrotu do odpowiedniej animacji po zakończeniu sekwencji ataku, co zapewniło zgodność pomiędzy logiką programu a prezentowaną animacją.

Przeprowadzono również testy dotyczące mechanizmu wykrywania krawędzi platform. Ich celem było wyeliminowanie sytuacji, w których przeciwnik opuszczał wyznaczony obszar patrolowania lub spadał z platformy. Po dostrojeniu parametrów odpowiedzialnych za wykrywanie podłoża uzyskano stabilne zachowanie postaci na wszystkich przygotowanych fragmentach planszy.

Istotną zaletą zastosowanego rozwiązania okazała się możliwość regulacji większości parametrów bez konieczności modyfikacji kodu źródłowego. Wartości odpowiadające za prędkość ruchu, zasięg wykrywania, częstotliwość ataków oraz liczbę punktów życia mogły być zmieniane bezpośrednio z poziomu Inspektora Unity. Pozwoliło to znacząco skrócić czas testowania kolejnych konfiguracji oraz ułatwiło dobór parametrów zapewniających odpowiedni poziom trudności gry.

Po wprowadzeniu opisanych poprawek testy potwierdziły poprawność działania zaimplementowanych mechanizmów oraz wykazały, że zastosowany model sterowania oparty na skończonym automacie stanów spełnia założenia przyjęte na etapie projektowania systemu sztucznej inteligencji.

2.5 Wnioski z realizacji projektu

Realizacja części praktycznej pozwoliła zweryfikować zagadnienia przedstawione w teoretycznej części pracy. Implementacja systemu sztucznej inteligencji wykazała, że nawet stosunkowo proste modele decyzyjne mogą zapewnić czytelne i wiarygodne zachowanie postaci niezależnych, pod warunkiem ich odpowiedniego zaprojektowania oraz dostrojenia.

Najważniejszym elementem projektu okazało się zastosowanie skończonych automatów stanów jako podstawowego mechanizmu sterowania zachowaniem przeciwników.

Rozwiązanie to umożliwiło stworzenie czytelnej architektury programu, w której

poszczególne stany odpowiadają konkretnym zachowaniom postaci. Dzięki temu implementacja była stosunkowo prosta, a jednocześnie pozwalała na łatwe rozszerzanie funkcjonalności o kolejne elementy bez konieczności przebudowy całego systemu.

Istotną zaletą zastosowanego podejścia było również rozdzielenie odpowiedzialności pomiędzy poszczególne komponenty programu. Oddzielenie logiki sterującej przeciwnikami od mechanizmów odpowiedzialnych za punkty życia, animacje oraz interfejs użytkownika zwiększyło przejrzystość projektu i ułatwiło jego rozwój. Rozwiązanie to pozwoliło także na ponowne wykorzystanie części komponentów podczas implementacji różnych typów przeciwników.

Realizacja projektu pokazała również, że największe trudności nie wynikały z implementacji samego automatu stanów, lecz z konieczności zapewnienia poprawnej współpracy pomiędzy logiką programu, systemem animacji oraz fizyką silnika Unity. Odpowiednia synchronizacja tych elementów wymagała wielokrotnego testowania oraz wprowadzania poprawek eliminujących błędy pojawiające się podczas rozgrywki.

Opracowany system spełnił założenia przyjęte w podrozdziale 2.1. Przeciwnicy reagują na obecność gracza, realizują przypisane im zachowania, uczestniczą w systemie walki oraz współpracują z pozostałymi elementami gry. Jednocześnie architektura projektu pozostawia możliwość dalszej rozbudowy o kolejne typy przeciwników, nowe stany automatu lub bardziej zaawansowane mechanizmy podejmowania decyzji.

W przyszłości projekt mógłby zostać rozszerzony między innymi o implementację drzew behawioralnych, systemów planowania celów lub elementów uczenia maszynowego, co umożliwiłoby porównanie skuteczności różnych metod sterowania postaciami niezależnymi opisanych w podrozdziale 1.3. Interesującym kierunkiem rozwoju byłoby również zwiększenie liczby przeciwników oraz rozbudowanie ich współpracy w ramach wspólnego systemu sztucznej inteligencji.

Podsumowując, zrealizowany projekt potwierdził, że odpowiednio zaprojektowany system oparty na skończonych automatach stanów jest rozwiązaniem wystarczającym do stworzenia funkcjonalnej sztucznej inteligencji dla gry platformowej 2D. Jednocześnie doświadczenia zdobyte podczas implementacji pozwoliły zweryfikować wiedzę przedstawioną w części

teoretycznej oraz pokazać, że o powodzeniu implementacji decyduje nie złożoność samego algorytmu decyzyjnego, lecz jakość jego integracji z pozostałymi systemami silnika — animacją, fizyką i warstwą prezentacji.

3 Techniki sztucznej inteligencji stosowane w programowaniu NPC

W poprzednim rozdziale przedstawiono proces projektowania oraz implementacji systemu sztucznej inteligencji wykorzystanego w opracowanej grze platformowej 2D. Zastosowane rozwiązania zostały dobrane z uwzględnieniem założeń projektowych oraz możliwości środowiska Unity, co pozwoliło na stworzenie funkcjonalnego systemu sterowania zachowaniem postaci niezależnych. Niniejszy rozdział przedstawia szczegółową analizę technik przedstawionych w podrozdziale 1.3 na poziomie mechaniki ich działania.

Poszczególne rozwiązania omówiono pod kątem ich zasady działania, budowy, zalet oraz ograniczeń, a także możliwości wykorzystania podczas tworzenia gier komputerowych. W miejscach, w których ma to uzasadnienie, odniesiono się również do rozwiązań zastosowanych w części praktycznej pracy, wskazując powody ich wykorzystania lub świadomej rezygnacji z implementacji. Kolejno omówiono maszyny stanów, drzewa behawioralne, sieci neuronowe wraz z uczeniem maszynowym, systemy regułowe i logikę rozmytą oraz algorytmy planowania ścieżek.

3.1 Maszyny stanów (Finite-State Machines)

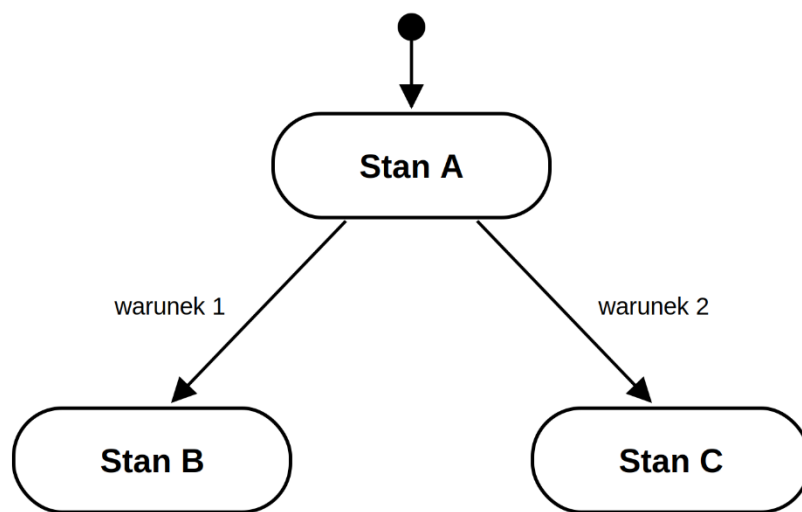
Jedną z najczęściej wykorzystywanych technik sztucznej inteligencji w grach komputerowych są skończone maszyny stanów (Finite-State Machines – FSM), określane również jako automaty stanów. Rozwiązanie to polega na modelowaniu zachowania postaci jako zbioru jasno zdefiniowanych stanów oraz reguł określających moment przejścia pomiędzy nimi. W danym momencie postać może znajdować się wyłącznie w jednym stanie, a zmiana jej zachowania następuje dopiero po spełnieniu określonych warunków przejścia. Dzięki temu logika działania NPC jest uporządkowana, przewidywalna oraz stosunkowo łatwa do implementacji i testowania.³⁴

Formalnie automat stanów można opisać jako strukturę składającą się z pięciu podstawowych elementów:

³⁴ M. Buckland, *Programming Game AI by Example*, Wordware Publishing, Plano 2005..

- zbioru stanów,
- zbioru przejść pomiędzy stanami,
- warunków aktywujących przejścia,
- stanu początkowego,
- działań wykonywanych w poszczególnych stanach.

Rysunek 3.1 przedstawia ogólną ideę działania skończonego automatu stanów.



Rysunek 3.1. Ogólna struktura maszyny stanów.

Źródło: opracowanie własne.

W praktyce działania gry automat stanów jest wykonywany cyklicznie podczas każdej iteracji głównej pętli programu. W pierwszej kolejności analizowane są warunki przejścia pomiędzy stanami, a następnie wykonywana jest logika przypisana do aktualnie aktywnego stanu. Taki sposób działania powoduje, że zachowanie przeciwnika może dynamicznie zmieniać się wraz ze zmianą sytuacji panującej w świecie gry, przy jednoczesnym zachowaniu pełnej kontroli nad przebiegiem procesu decyzyjnego.³⁵

³⁵ R. Nystrom, *Game Programming Patterns*, Genever Benning 2014, rozdz. „State”; por. I. Millington, J. Funge, *Artificial Intelligence for Games*, CRC Press.

Poza logiką wykonywaną cyklicznie w trakcie trwania stanu, maszyny stanów zwykle definiują również akcje uruchamiane jednorazowo przy wejściu do stanu oraz przy jego opuszczeniu. Akcja wejścia pozwala przygotować postać do nowego zachowania, na przykład uruchomić odpowiednią animację lub wyzerować licznik czasu, natomiast akcja wyjścia umożliwia uporządkowanie stanu poprzedniego, choćby przerwanie trwającej animacji. Rozdzielenie tych trzech momentów zwiększa przewidywalność działania automatu i ułatwia synchronizację logiki z warstwą wizualną. Schemat działania automatu można przedstawić w uproszczonej postaci za pomocą następującego pseudokodu:

```
while (gra działa)
{
    sprawdź warunki przejścia dla bieżącego stanu;

    jeżeli warunek został spełniony
    {
        wykonaj akcje kończące stan bieżący;
        ustaw nowy stan jako bieżący;
        wykonaj akcje inicjujące nowy stan;
    }
    wykonaj logikę bieżącego stanu;
}
```

Przedstawiony mechanizm jest stosunkowo prosty, jednak jego skuteczność wynika z jednoznacznego rozdzielenia odpowiedzialności pomiędzy poszczególne stany. Każdy z nich realizuje wyłącznie określony fragment zachowania, natomiast decyzja o zmianie stanu podejmowana jest na podstawie wcześniej zdefiniowanych reguł. Takie podejście ogranicza liczbę wzajemnych zależności pomiędzy poszczególnymi elementami systemu oraz ułatwia jego rozwój.³⁶

³⁶ I. Millington, J. Funge, dz. cyt.

Do najważniejszych zalet automatów stanów należy przede wszystkim prostota implementacji, niewielkie wymagania obliczeniowe oraz łatwość debugowania. Dzięki jednoznacznie określonym stanom programista może stosunkowo łatwo prześledzić przebieg działania algorytmu i zidentyfikować źródło ewentualnych błędów. Rozwiązanie to dobrze sprawdza się również podczas projektowania przeciwników o ograniczonej liczbie możliwych zachowań, co powoduje, że od wielu lat pozostaje jednym z najczęściej wykorzystywanych mechanizmów sztucznej inteligencji w grach komputerowych.³⁷

Wraz ze wzrostem liczby stanów oraz możliwych przejść pojawia się jednak problem gwałtownego zwiększania złożoności automatu. Każdy nowy stan może wymagać zdefiniowania przejść do wielu pozostałych stanów, co utrudnia rozwój projektu oraz zwiększa ryzyko występowania błędów. Zjawisko to określane jest często jako eksplozja przejść (transition explosion). W celu ograniczenia tego problemu opracowano hierarchiczne maszyny stanów (Hierarchical Finite-State Machines – HFSM), w których wybrane stany mogą zawierać własne podstany. Pozwala to grupować podobne zachowania oraz znacząco zmniejszyć liczbę wymaganych przejść pomiędzy elementami systemu.

W opracowanym w ramach niniejszej pracy projekcie wykorzystano klasyczne maszyny stanów do sterowania zachowaniem obu typów przeciwników. Szczegółowy opis zastosowanych automatów wraz z diagramami stanów przedstawiono w rozdziale 2 (rys. 2.1 oraz rys. 2.2), natomiast uzasadnienie wyboru tej techniki omówiono w podrozdziale 2.2.

³⁷ M. Buckland, dz. cyt.

3.2 Drzewa behawioralne (Behavior Trees)

Drzewa behawioralne (Behavior Trees – BT) stanowią jedną z najczęściej stosowanych metod projektowania sztucznej inteligencji w nowoczesnych grach komputerowych. Technika ta została opracowana jako rozwinięcie klasycznych automatów stanów, których złożoność znacząco wzrasta wraz z liczbą możliwych zachowań oraz przejść pomiędzy nimi. Głównym założeniem drzew behawioralnych jest hierarchiczna organizacja logiki decyzyjnej, dzięki której zachowania postaci mogą być tworzone z niewielkich, niezależnych elementów wielokrotnego użytku.³⁸

W przeciwieństwie do automatów stanów, w których w danym momencie aktywny jest wyłącznie jeden stan, drzewo behawioralne składa się z wielu połączonych węzłów tworzących strukturę przypominającą drzewo. Węzeł znajdujący się na najwyższym poziomie, określany jako korzeń (root), rozpoczyna proces podejmowania decyzji, przekazując sterowanie kolejnym elementom drzewa. Każdy z nich odpowiada za określony fragment logiki, dzięki czemu całość może być łatwo rozbudowywana oraz modyfikowana bez konieczności przebudowy całego systemu.³⁹

Podstawowymi elementami drzewa behawioralnego są węzły sterujące oraz węzły wykonawcze. Do pierwszej grupy zaliczane są przede wszystkim sekwencje (Sequence) oraz selektory (Selector). Sekwencja wykonuje swoje węzły potomne kolejno i kończy działanie sukcesem wyłącznie wtedy, gdy wszystkie operacje zakończą się powodzeniem. W

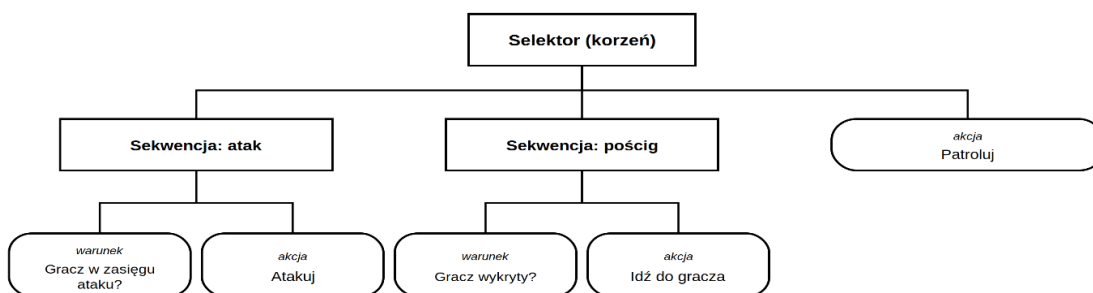
³⁸ M. Colledanchise, P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*, CRC Press, Boca Raton 2018.

³⁹ M. Colledanchise, P. Ögren, dz. cyt.

przypadku niepowodzenia któregoś z elementów wykonanie zostaje natychmiast przerwane. Odwrotnie działa selektor, którego zadaniem jest wyszukiwanie pierwszego poprawnego rozwiązania. Jeżeli jeden z potomków zakończy działanie sukcesem, pozostałe nie są już wykonywane.

Istotnym elementem drzew behawioralnych są również dekoratory (Decorator Nodes), które modyfikują sposób działania pojedynczych gałęzi drzewa. Mogą one odwracać wynik działania wybranego węzła, ograniczać liczbę jego wywołań lub uzależniać wykonanie od spełnienia określonych warunków. Liście drzewa (Leaf Nodes) stanowią natomiast końcowe elementy struktury i odpowiadają za wykonywanie konkretnych akcji, takich jak ruch postaci, rozpoczęcie ataku czy sprawdzenie odległości od gracza.⁴⁰

Ogólną strukturę drzewa behawioralnego przedstawiono na rysunku 3.2.



Rysunek 3.2. Przykładowa struktura drzewa behawioralnego.

Źródło: opracowanie własne.

Działanie drzewa behawioralnego opiera się na cyklicznym przetwarzaniu jego struktury, określanym jako tick. Podczas każdej iteracji gry wykonywana jest analiza rozpoczynająca się od węzła głównego i przechodząca przez kolejne gałęzie zgodnie z ich logiką działania. Każdy odwiedzony węzeł zwraca jeden z trzech możliwych statusów wykonania:

- Success – zadanie zostało wykonane poprawnie,
- Failure – wykonanie zakończyło się niepowodzeniem,
- Running – zadanie nadal trwa i powinno być kontynuowane podczas następnego przebiegu drzewa.

⁴⁰ I. Millington, J. Funge, dz. cyt.

Mechanizm ten umożliwia płynne wykonywanie działań rozłożonych na wiele klatek, takich jak poruszanie się postaci, oczekiwanie na zakończenie animacji czy śledzenie celu. Dzięki temu zachowania NPC są bardziej elastyczne niż w przypadku klasycznych automatów stanów.⁴¹

Największą zaletą drzew behawioralnych jest ich wysoka modularność. Poszczególne fragmenty logiki mogą być wykorzystywane wielokrotnie przez różne typy przeciwników, co znacząco ogranicza liczbę powtarzającego się kodu. Struktura drzewa ułatwia również współpracę programistów i projektantów poziomów, ponieważ zachowania mogą być tworzone w graficznych edytorach dostępnych w wielu współczesnych silnikach gier, między innymi w Unreal Engine. W porównaniu z automatami stanów rozwiązanie to lepiej radzi sobie z rozbudowanymi zachowaniami, w których liczba możliwych decyzji jest bardzo duża. Zalety te sprawiły, że drzewa behawioralne znalazły swoje zastosowanie w komercyjnych produkcjach – za tytuł który spopularyzował tę technikę, uznaje się grę Halo 2 (2004), gdzie posłużyła ona do sterowania złożonym zachowaniem przeciwników.⁴²

Drzewa behawioralne posiadają jednak również pewne ograniczenia. Ich implementacja jest bardziej złożona niż w przypadku klasycznych automatów stanów, a niewłaściwie zaprojektowana struktura może prowadzić do trudności związanych z analizą przebiegu wykonywania oraz diagnozowaniem błędów. W małych projektach wykorzystujących jedynie kilka prostych zachowań korzyści wynikające z zastosowania tej techniki często nie rekompensują dodatkowego stopnia skomplikowania implementacji.

W projekcie opracowanym na potrzeby niniejszej pracy drzewa behawioralne nie zostały wykorzystane. Decyzja ta wynikała przede wszystkim z ograniczonej skali gry oraz niewielkiej liczby zachowań implementowanych dla dwóch typów przeciwników. Zastosowanie automatów stanów umożliwiło uzyskanie czytelnej struktury kodu, łatwiejsze testowanie oraz pełną kontrolę nad przebiegiem procesu decyzyjnego. W przypadku dalszej rozbudowy projektu o większą liczbę przeciwników oraz bardziej złożone zależności pomiędzy

⁴¹ M. Colledanchise, P. Ögren, dz. cyt.

⁴² D. Isla, *Handling Complexity in the Halo 2 AI*, Game Developers Conference, San Francisco 2005.

zachowaniami zastosowanie drzew behawioralnych mogłoby jednak stanowić uzasadniony kierunek dalszego rozwoju systemu sztucznej inteligencji.

3.3 Sieci neuronowe i uczenie maszynowe

Odmienną grupę technik sztucznej inteligencji stanowią metody oparte na uczeniu maszynowym (machine learning), w tym sztuczne sieci neuronowe. W przeciwieństwie do omówionych wcześniej automatów stanów oraz drzew behawioralnych, w których cała logika zachowania jest jawnie zdefiniowana przez twórcę, w podejściu opartym na uczeniu maszynowym zachowanie postaci nie jest programowane wprost, lecz wyłania się z procesu treningu prowadzonego na danych lub w wyniku interakcji ze środowiskiem gry. Projektant określa jedynie cel oraz sposób oceny działania agenta, natomiast konkretna strategia postępowania wypracowywana jest automatycznie.⁴³

Sztuczna sieć neuronowa stanowi model obliczeniowy inspirowany budową biologicznego układu nerwowego. Składa się z połączonych ze sobą jednostek nazywanych neuronami, zorganizowanych w warstwy: wejściową, jedną lub więcej warstw ukrytych oraz warstwę wyjściową. Każde połączenie pomiędzy neuronami posiada przypisaną wagę, a proces uczenia polega na stopniowym dostrajaniu tych wag w taki sposób, aby sieć generowała pożądane wyniki dla zadanych danych wejściowych. W kontekście gier komputerowych dane wejściowe mogą opisywać stan świata gry — na przykład odległość od gracza, poziom zdrowia postaci czy obecność przeszkód — natomiast wyjście sieci reprezentuje decyzję o podjęciu określonego działania.⁴⁴

⁴³ G. N. Yannakakis, J. Togelius, *dz. cyt.*

⁴⁴ I. Millington, J. Funge, *dz. cyt.*

W programowaniu postaci niezależnych szczególne znaczenie ma uczenie ze wzmocnieniem (reinforcement learning). W modelu tym agent podejmuje działania w środowisku gry i otrzymuje sygnał nagrody lub kary w zależności od skutków swoich decyzji. Celem procesu uczenia jest wypracowanie strategii postępowania (policy) maksymalizującej łączną nagrodę uzyskiwaną w dłuższym horyzoncie czasu. Dzięki temu agent może samodzielnie odkrywać skuteczne wzorce zachowania, które nie zostały bezpośrednio zaprogramowane, a niekiedy również rozwiązania nieoczywiste z perspektywy projektanta. Podejście to bywa wykorzystywane w badaniach nad grami wymagającymi długofalowego planowania oraz reagowania na zachowanie przeciwnika.⁴⁵

Praktycznym narzędziem umożliwiającym stosowanie uczenia maszynowego w środowisku Unity jest pakiet ML-Agents, pozwalający trenować agentów bezpośrednio wewnątrz silnika. Rozwiązanie to udostępnia komponenty odpowiedzialne za obserwację stanu środowiska, wykonywanie akcji oraz przyznawanie nagród, a także integruje się z zewnętrznymi bibliotekami uczenia głębokiego wykorzystywanymi podczas treningu. Umożliwia to prowadzenie eksperymentów z zachowaniami adaptacyjnymi bez konieczności budowania całego potoku uczenia od podstaw.⁴⁶

Najważniejszą zaletą podejścia opartego na uczeniu maszynowym jest zdolność do adaptacji oraz tworzenia zachowań emergentnych, to znaczy takich, które nie wynikają bezpośrednio z pojedynczych reguł, lecz z ich wzajemnego oddziaływania w trakcie treningu. Rozwiązania te sprawdzają się przede wszystkim tam, gdzie liczba możliwych sytuacji jest zbyt duża, aby opisać ją w sposób jawny, lub gdy pożądane jest, aby zachowanie postaci było trudne do przewidzenia i dostosowywało się do stylu gry użytkownika.

Jednocześnie metody te wiążą się z istotnymi ograniczeniami, które w praktyce produkcyjnej znacząco zawężają ich zastosowanie. Proces treningu jest kosztowny obliczeniowo i czasochłonny, a jego rezultat zależy od jakości przygotowanych danych oraz właściwego

⁴⁵ R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, wyd. 2, MIT Press, Cambridge 2018.

⁴⁶ Unity Technologies, *Unity ML-Agents Toolkit Documentation*, <https://unity-technologies.github.io/ml-agents/>, [dostęp: 20.08.2026].

zdefiniowania funkcji nagrody. Poważnym wyzwaniem pozostaje również ograniczona interpretowalność decyzji wytrenowanego modelu — w odróżnieniu od automatu stanów, w którym można jednoznacznie wskazać przyczynę przejścia do danego stanu, sieć neuronowa działa w dużej mierze jak „czarna skrzynka”. Utrudnia to testowanie oraz kontrolę balansu rozgrywki i zwiększa ryzyko wystąpienia zachowań niepożądanych z punktu widzenia projektanta. Z tych powodów w wielu gatunkach gier klasyczne, w pełni jawne techniki nadal pozostają rozwiązaniem podstawowym, a uczenie maszynowe traktowane jest jako uzupełnienie stosowane tam, gdzie jego zalety rzeczywiście przeważają nad kosztami.⁴⁷

W projekcie zrealizowanym na potrzeby niniejszej pracy świadomie zrezygnowano z zastosowania uczenia maszynowego. Decyzja ta wynikała zarówno z niewielkiej skali projektu, jak i z przyjętego celu, którym była czytelna i przewidywalna implementacja zachowań dwóch typów przeciwników. Wykorzystanie modeli uczących się nie przyniosłoby w tym przypadku istotnych korzyści, jednocześnie znacząco komplikując proces implementacji, testowania oraz strojenia zachowań, co szerzej uzasadniono w podrozdziale 2.2.

⁴⁷ G. N. Yannakakis, J. Togelius, dz. cyt.

3.4 Systemy oparte na regułach i logice rozmytej

Kolejną grupę technik stosowanych w programowaniu postaci niezależnych stanowią systemy regułowe oraz logika rozmyta. Obie metody łączy sposób opisu zachowania w postaci zestawu reguł wiążących warunki panujące w świecie gry z podejmowanymi działaniami, różnią się jednak sposobem, w jaki traktują spełnienie tych warunków.⁴⁸

W klasycznym systemie regułowym zachowanie postaci definiowane jest za pomocą zbioru reguł typu „jeżeli warunek, to działanie” (if–then). System analizuje aktualny stan świata gry i wykonuje te reguły, których warunki zostały spełnione. Podejście to bywa nazywane systemem produkcyjnym, a jego pokrewną formą są systemy eksperckie, w których wiedza o sposobie postępowania zapisana jest właśnie w postaci reguł. Największą zaletą tego rozwiązania jest przejrzystość oraz łatwość implementacji — zależność pomiędzy sytuacją a reakcją postaci jest bezpośrednia i czytelna, a działanie systemu można w prosty sposób prześledzić i przetestować.⁴⁹

Ograniczeniem systemów regułowych jest ich słaba skalowalność. Wraz ze wzrostem liczby uwzględnianych sytuacji rośnie liczba reguł, a wzajemne zależności pomiędzy nimi stają się coraz trudniejsze do utrzymania. Pojawia się również ryzyko powstawania reguł wzajemnie sprzecznych lub pokrywających się, co utrudnia przewidzenie zachowania systemu. Z tego względu podejście to sprawdza się najlepiej przy stosunkowo niewielkiej liczbie jasno określonych zachowań, natomiast w bardziej rozbudowanych systemach decyzyjnych zwykle łączone jest z innymi technikami, takimi jak automaty stanów czy drzewa behawioralne.

⁴⁸ M. Buckland, dz. cyt.

⁴⁹ I. Millington, J. Funge, dz. cyt.

Istotnym rozszerzeniem klasycznego podejścia regułowego jest logika rozmyta (fuzzy logic). Wynika ona z obserwacji, że wiele pojęć opisujących stan świata gry ma charakter stopniowalny, a nie zerojedynekowy. W logice klasycznej warunek taki jak „gracz jest blisko” przyjmuje wyłącznie wartość prawdy lub fałszu, przez co niewielka zmiana odległości może spowodować gwałtowną zmianę zachowania postaci. Logika rozmyta pozwala natomiast wyrazić stopień przynależności danej wartości do zbioru — odległość może być na przykład „blisko” w 70 procentach i jednocześnie „daleko” w 30 procentach — dzięki czemu przejścia pomiędzy zachowaniami stają się płynniejsze i bardziej naturalne.⁵⁰

Proces podejmowania decyzji w systemie opartym na logice rozmytej przebiega w trzech etapach. W pierwszym, nazywanym rozmywaniem (fuzzification), wartości liczbowe opisujące stan świata gry — takie jak odległość, poziom zdrowia czy zapas amunicji — przekształcane są na zmienne lingwistyczne z określonym stopniem przynależności, na przykład „niski”, „średni” lub „wysoki”. Następnie stosowany jest zbiór reguł rozmytych, które na podstawie tych zmiennych określają zalecane działanie postaci. W ostatnim etapie, nazywanym wyostrzaniem (defuzzification), rozmyty wynik przekształcany jest z powrotem na konkretną wartość liczbową sterującą zachowaniem NPC. Dzięki takiemu mechanizmowi postać może płynnie regulować, na przykład, poziom agresji lub skłonność do ucieczki zamiast skokowo przełączać się pomiędzy skrajnymi trybami działania.

W praktyce projektowej logika rozmyta bywa wykorzystywana między innymi do wyboru celu lub broni, oceny atrakcyjności możliwych działań czy modelowania parametrów zachowania, które powinny zmieniać się w sposób ciągły. Jej zaletą jest większa naturalność i elastyczność decyzji, wadą natomiast — trudniejsze strojenie oraz mniejsza przejrzystość niż w przypadku prostych reguł logicznych, ponieważ wynik zależy od kształtu funkcji przynależności oraz sposobu łączenia reguł.

W projekcie towarzyszącym niniejszej pracy nie zastosowano logiki rozmytej, a zachowanie przeciwników oparto na jawnych warunkach logicznych związanych przede wszystkim z

⁵⁰ M. Buckland, dz. cyt.

progami odległości od gracza. Warto jednak zauważyć, że zastosowany mechanizm dwóch niezależnych progów wykrywania i porzucania pościgu, opisany w podrozdziale 2.3, pełni funkcję zbliżoną do jednego z celów logiki rozmytej — eliminuje gwałtowne, wielokrotne przełączanie stanów w pobliżu granicy zasięgu, poprawiając stabilność i czytelność zachowania postaci przy zachowaniu pełnej przewidywalności reguł.

3.5 Planowanie ścieżek i algorytmy nawigacyjne

Ostatnią z omawianych grup technik stanowią algorytmy planowania ścieżek (pathfinding), odpowiedzialne za wyznaczanie trasy przemieszczenia postaci niezależnej pomiędzy dwoma punktami świata gry z uwzględnieniem przeszkód oraz ograniczeń terenu. W odróżnieniu od wcześniej omówionych technik, które dotyczyły przede wszystkim podejmowania decyzji o wyborze zachowania, planowanie ścieżek odpowiada za sposób realizacji ruchu, stanowiąc uzupełnienie systemów decyzyjnych.

Podstawą działania większości algorytmów planowania ścieżek jest reprezentacja świata gry w postaci grafu, którego wierzchołki odpowiadają możliwym do zajęcia pozycjom, a krawędzie — dopuszczalnym przejściom pomiędzy nimi. W grach dwuwymiarowych często stosuje się siatki kwadratowe lub sieci punktów kontrolnych (waypoints), natomiast w grach trójwymiarowych powszechnym rozwiązaniem są siatki nawigacyjne (navmesh), opisujące obszary możliwe do przejścia w postaci połączonych wielokątów. Wybór reprezentacji ma bezpośredni wpływ na dokładność wyznaczanych tras oraz koszt obliczeniowy algorytmu.⁵¹

Jednym z najprostszych algorytmów wyszukiwania ścieżki jest algorytm Dijkstry, który wyznacza najkrótsze trasy od punktu początkowego do wszystkich pozostałych wierzchołków grafu, przeszukując je w kolejności rosnącego kosztu dojścia. Zapewnia on znalezienie optymalnej ścieżki, jednak przeszukuje graf bez uwzględnienia położenia celu, przez co bywa nieefektywny w dużych środowiskach.⁵²

⁵¹ I. Millington, J. Funge, dz. cyt.

⁵² M. Buckland, dz. cyt.

Rozwinięciem tej idei jest algorytm A* (A-star), który stanowi obecnie standard w wyznaczaniu ścieżek w grach komputerowych. Łączy on rzeczywisty koszt dojścia do danego wierzchołka z heurystycznym oszacowaniem odległości pozostałej do celu, co pozwala ukierunkować przeszukiwanie i znacząco ograniczyć liczbę analizowanych wierzchołków. Wartość każdego wierzchołka wyznaczana jest jako suma kosztu przebytej drogi oraz oszacowania kosztu drogi pozostałej. Kluczowe znaczenie ma dobór funkcji heurystycznej — aby algorytm gwarantował znalezienie ścieżki optymalnej, heurystyka nie może przeszacowywać rzeczywistej odległości do celu. W praktyce, w zależności od charakteru siatki, stosuje się między innymi metrykę Manhattan lub odległość euklidesową.⁵³

Wyznaczona ścieżka opisuje jedynie ogólny przebieg trasy, dlatego jej realizacja wymaga uzupełnienia o mechanizmy sterowania ruchem na poziomie lokalnym. Odpowiadają za to techniki określane jako zachowania sterujące (steering behaviors), obejmujące między innymi podążanie za ścieżką, unikanie przeszkód oraz płynne dostosowywanie prędkości i kierunku ruchu. Współczesne silniki gier, w tym Unity, udostępniają gotowe systemy nawigacji oparte na siatkach navmesh wraz z komponentami agentów, dzięki czemu twórca nie musi samodzielnie implementować algorytmów wyszukiwania ścieżek.

Znaczenie algorytmów planowania ścieżek zależy w dużym stopniu od charakteru gry. W produkcjach z rozbudowanymi, otwartymi światami stanowią one podstawę poruszania się postaci niezależnych, natomiast w prostszych grach dwuwymiarowych ich zastosowanie często nie jest konieczne, a ruch postaci można oprzeć na uproszczonych mechanizmach reagujących na najbliższe otoczenie.

W projekcie zrealizowanym w ramach niniejszej pracy nie wykorzystano algorytmów planowania ścieżek ani systemu navmesh. Ze względu na dwuwymiarowy charakter gry oraz prostą konstrukcję planszy ruch przeciwników oparto na patrolu ograniczonym zasięgiem oraz na wykrywaniu krawędzi platform za pomocą promienia kontrolnego skierowanego w dół, co opisano w podrozdziałach 2.2 oraz 2.3. Rozwiązanie takie okazało się w pełni

⁵³ S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, wyd. 4, Pearson, Harlow 2021.

wystarczające do realizacji przyjętych założeń projektowych, przy jednoczesnym ograniczeniu złożoności implementacji oraz kosztów obliczeniowych.

4 Zakończenie

Niniejsza praca poświęcona była projektowaniu oraz implementacji postaci niezależnych w dwuwymiarowej grze platformowej z wykorzystaniem sztucznej inteligencji. Połączono w niej część teoretyczną, obejmującą przegląd technologii, algorytmów oraz modeli stosowanych przy programowaniu NPC, z częścią praktyczną, w której zaprojektowano, zaimplementowano oraz przetestowano zachowania dwóch typów przeciwników w środowisku Unity. W niniejszym rozdziale zebrano najważniejsze wnioski wynikające z realizacji pracy oraz zweryfikowano postawione na jej wstępie tezy.

4.1 Podsumowanie wniosków

Realizacja pracy pozwoliła osiągnąć postawiony cel, którym było przedstawienie procesu projektowania i implementacji postaci niezależnych sterowanych sztuczną inteligencją oraz połączenie wiedzy teoretycznej z praktycznym wdrożeniem w konkretnym projekcie. Część

teoretyczna dostarczyła podstaw niezbędnych do świadomego doboru rozwiązań, natomiast część praktyczna umożliwiła ich weryfikację w działającym prototypie gry.

W części teoretycznej wykazano, że sztuczna inteligencja w grach komputerowych nie jest tożsama ze sztuczną inteligencją w rozumieniu ogólnym, lecz stanowi zbiór metod służących tworzeniu wiarygodnych i użytecznych zachowań w świecie gry. Kluczowym wnioskiem tej części pracy jest spostrzeżenie, że o jakości sztucznej inteligencji w grach decyduje nie optymalność podejmowanych decyzji, lecz ich czytelność, spójność oraz zgodność z zasadami świata gry. Przegląd technologii, modeli decyzyjnych oraz technik szczegółowo omówionych w rozdziale trzecim pokazał ponadto, że dobór rozwiązania powinien wynikać przede wszystkim ze skali i charakteru produkcji, a nie z samej dostępności coraz bardziej zaawansowanych narzędzi.

Część praktyczna potwierdziła te obserwacje na przykładzie konkretnego projektu. Do sterowania zachowaniem obu typów przeciwników wykorzystano skończone automaty stanów, które — mimo swojej prostoty — okazały się w pełni wystarczające do uzyskania czytelnych i przewidywalnych zachowań. Zastosowanie architektury komponentowej, w której logika decyzyjna, obsługa punktów życia oraz warstwa interfejsu i animacji zostały rozdzielone na niezależne moduły, zwiększyło przejrzystość projektu, umożliwiło ponowne wykorzystanie kodu oraz ułatwiło testowanie poszczególnych mechanizmów.

Istotnym wnioskiem wynikającym z realizacji projektu jest spostrzeżenie, że największe trudności nie wynikały z implementacji samego algorytmu decyzyjnego, lecz z konieczności zapewnienia poprawnej współpracy pomiędzy logiką programu, systemem animacji oraz fizyką silnika. To właśnie synchronizacja tych elementów — widoczna między innymi w problemie wielokrotnego wyzwalania animacji ataku oraz w konieczności precyzyjnego zgrania momentu zadawania obrażeń z klatką animacji — wymagała najwięcej pracy i wielokrotnego testowania. Wniosek ten ma charakter uniwersalny i pozostaje aktualny niezależnie od zastosowanej techniki sterowania zachowaniem postaci.

Świadome ograniczenie zakresu projektu — rezygnacja z uczenia maszynowego, systemów nawigacji opartych na siatkach navmesh oraz drzew behawioralnych — nie było wynikiem

braków technicznych, lecz decyzją projektową spójną z wnioskami części teoretycznej. W przypadku gry o niewielkiej skali techniki te nie przyniosłyby istotnych korzyści, zwiększyłyby natomiast złożoność implementacji oraz koszt testowania. Potwierdza to tezę o nadrzędności doboru rozwiązania względem skali produkcji nad dążeniem do maksymalnej złożoności systemu.

Podsumowując, praca wykazała, że nawet stosunkowo proste modele decyzyjne pozwalają uzyskać funkcjonalną i wiarygodną sztuczną inteligencję dla gry platformowej 2D, pod warunkiem ich odpowiedniego zaprojektowania, dostrojenia oraz integracji z pozostałymi systemami gry. Realizacja projektu połączyła pogłębienie wiedzy teoretycznej ze zdobyciem praktycznych umiejętności programistycznych, co stanowi zarówno wartość edukacyjną, jak i inżynierską niniejszej pracy.

4.2 Weryfikacja postawionych tez

We wstępie pracy przyjęto cztery tezy badawcze dotyczące zastosowania skończonych automatów stanów w projektowaniu postaci niezależnych, znaczenia integracji systemu decyzyjnego z pozostałymi elementami silnika, wpływu architektury komponentowej na jakość projektu oraz zasad doboru techniki sterowania zachowaniem postaci. Weryfikację przeprowadzono kolejno dla każdej z nich, w oparciu o rezultaty prac projektowych oraz przeprowadzonych badań własnych.

Pierwsza teza dotyczyła wystarczalności skończonych automatów stanów do uzyskania czytelnych i wiarygodnych zachowań NPC w grze o ograniczonej skali. Teza ta została potwierdzona w części praktycznej pracy. Zaimplementowane automaty stanów umożliwiły stworzenie dwóch wyraźnie różniących się typów przeciwników — patrolującego oraz strażnika — których zachowanie pozostaje przewidywalne i zrozumiałe dla gracza. Przeprowadzone testy, opisane w podrozdziale 2.4, wykazały poprawność przechodzenia pomiędzy stanami oraz stabilność działania systemu, a wprowadzenie mechanizmu dwóch niezależnych progów wykrywania wyeliminowało gwałtowne przełączanie się przeciwnika pomiędzy stanami. Uzyskane zachowania są spójne z zasadami świata gry i pozbawione

elementów sprawiających wrażenie nieuczciwych, co potwierdza założenia przyjęte w podrozdziale 2.1.

Druga teza zakładała, że o powodzeniu implementacji decyduje przede wszystkim jakość integracji systemu decyzyjnego z pozostałymi elementami silnika, a nie złożoność samego algorytmu. Teza ta również znalazła potwierdzenie w toku realizacji projektu. Sama implementacja automatu stanów okazała się stosunkowo prosta, natomiast najwięcej trudności przysporzyła poprawna współpraca logiki programu z systemem animacji oraz fizyką. To właśnie w tym obszarze pojawiały się błędy wymagające wielokrotnego testowania i wprowadzania poprawek, co jednoznacznie wskazuje, że kluczowym czynnikiem powodzenia była jakość integracji, a nie stopień skomplikowania mechanizmu decyzyjnego.

Trzecia teza dotyczyła wpływu architektury komponentowej na czytelność oraz łatwość rozbudowy projektu. Znalazła ona potwierdzenie w toku realizacji części praktycznej. Rozdzielenie logiki decyzyjnej, obsługi punktów życia oraz warstwy animacji na niezależne moduły umożliwiło ponowne wykorzystanie tych samych komponentów w obu typach przeciwników, a także pozwoliło testować poszczególne mechanizmy niezależnie od siebie. Modyfikacje wprowadzane w jednym module nie wymagały zmian w pozostałych, co potwierdza zasadność przyjętego rozwiązania.

Czwarta teza zakładała, że dobór techniki sterowania zachowaniem postaci powinien wynikać ze skali i charakteru produkcji. Analiza przeprowadzona w rozdziale trzecim wykazała, że techniki takie jak uczenie maszynowe, drzewa behawioralne czy algorytmy planowania ścieżek oferują możliwości znacznie wykraczające poza potrzeby zrealizowanego projektu, wiążąc się jednocześnie z istotnie wyższym kosztem implementacji, testowania oraz strojenia. Świadoma rezygnacja z tych rozwiązań na rzecz skończonych automatów stanów pozwoliła osiągnąć zakładany efekt przy zachowaniu pełnej kontroli nad zachowaniem postaci, co potwierdza słuszność przyjętego założenia.

Na podstawie przeprowadzonej analizy można stwierdzić, że postawione tezy zostały potwierdzone w całości. Zrealizowany projekt wykazał, że odpowiednio zaprojektowany

system oparty na skończonych automatach stanów stanowi rozwiązanie wystarczające do stworzenia funkcjonalnej sztucznej inteligencji dla gry platformowej 2D, a osiągnięte rezultaty pozostają zgodne z wnioskami sformułowanymi w części teoretycznej pracy. Jednocześnie architektura projektu została zaprojektowana w sposób umożliwiający jego dalszą rozbudowę o bardziej zaawansowane techniki opisane w rozdziale trzecim, co wyznacza naturalny kierunek ewentualnych prac rozwojowych.

BIBLIOGRAFIA

1. Buckland M., *Programming Game AI by Example*, Wordware Publishing, Plano 2005.
2. Colledanchise M., Ögren P., *Behavior Trees in Robotics and AI: An Introduction*, CRC Press, Boca Raton 2018.
3. Isla D., *Handling Complexity in the Halo 2 AI*, Game Developers Conference, San Francisco 2005.
4. Millington I., Funge J., *Artificial Intelligence for Games*, CRC Press, Boca Raton 2018.
5. Nystrom R., *Game Programming Patterns*, Genever Benning, 2014.
6. Orkin J., *Three States and a Plan: The AI of F.E.A.R.*, Game Developers Conference, San Francisco 2006.
7. Russell S. J., Norvig P., *Artificial Intelligence: A Modern Approach*, wyd. 4, Pearson, Harlow 2021.
8. Sutton R. S., Barto A. G., *Reinforcement Learning: An Introduction*, wyd. 2, MIT Press, Cambridge 2018.
9. Yannakakis G. N., Togelius J., *Artificial Intelligence and Games*, Springer, Cham 2018.

NETOGRAFIA

1. Pittman J., *The Pac-Man Dossier*, wersja 1.0.27, 2015, <https://pacman.holenet.info/>, [dostęp: 20.07.2026].
2. Unity Technologies, *Unity Manual — Animation Events*, <https://docs.unity3d.com/550/Documentation/Manual/animeditor-AnimationEvents.html>, [dostęp: 13.08.2026].
3. Unity Technologies, *Unity Manual — Animator Controller*, <https://docs.unity3d.com/2022.3/Documentation/Manual/class-AnimatorController.html>, [dostęp: 10.08.2026].

4. Unity Technologies, *Unity Manual — Gizmos*,
<https://docs.unity3d.com/6000.2/Documentation/Manual/gizmos-and-handles.html>,
[dostęp: 15.08.2026].
5. Unity Technologies, *Unity Manual — Navigation and Pathfinding*,
<https://docs.unity3d.com/Manual/Navigation.html>, [dostęp: 07.08.2026].
6. Unity Technologies, *Unity Manual — Physics 2D*,
<https://docs.unity3d.com/6000.5/Documentation/Manual/2d-physics/2d-physics.html>, [dostęp: 12.08.2026].
7. Unity Technologies, *Unity ML-Agents Toolkit Documentation*, <https://unity-technologies.github.io/ml-agents/>, [dostęp: 10.08.2026].
8. Unity Technologies, *Unity Scripting API — Physics2D.Raycast*,
<https://docs.unity3d.com/6000.5/Documentation/ScriptReference/Physics2D.Raycast.html>, [dostęp: 13.08.2026].

AKTY NORMATYWNE

1. Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych (RODO), Dz. Urz. UE L 119 z 4.05.2016.
2. Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2024/1689 z dnia 13 czerwca 2024 r. ustanawiające zharmonizowane przepisy dotyczące sztucznej inteligencji, Dz. Urz. UE L 2024/1689.
3. Ustawa z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych, Dz.U. 1994 nr 24 poz. 83 ze zm.

ZAŁĄCZNIK 1. SPIS ZAWARTOŚCI NOŚNIKA ELEKTRONICZNEGO

Kod źródłowy projektu został umieszczony w repozytorium w serwisie GitHub pod adresem:

<https://github.com/Wonszeek/Praca-Inzynierska>

W skład załączonego do pracy archiwum elektronicznego wchodzi projekt gry wykonany w środowisku Unity oraz wersja elektroniczna pracy. Poniżej przedstawiono szczegółowy wykaz elementów.

1. Projekt gry

Katalog /Assets/Scripts — kod źródłowy aplikacji napisany w języku C#, obejmujący skrypty odpowiedzialne za logikę decyzyjną przeciwników, sterowanie postacią gracza, obsługę punktów życia oraz warstwę interfejsu użytkownika.

Katalog /Assets/Scenes — sceny gry, w tym scena poziomu wykorzystana podczas testów.

Katalog /Assets/Prefabs — prefabrykaty postaci niezależnych oraz pozostałych obiektów świata gry.

Katalog /Assets/Animations — kontrolery animacji (Animator Controller) oraz klipy animacyjne przypisane do postaci.

Katalog /Assets/Pixel2DCastleTileset/Sprites oraz /Assets/Knight Files — zasoby graficzne wykorzystane w projekcie, w tym tekstury otoczenia i tileset zamku oraz grafiki postaci gracza.

Katalog /Assets/Tiles — zasoby oraz konfiguracja siatki kafelków (Tilemap) wykorzystanej do budowy poziomu.

2. Wersja elektroniczna pracy

Plik w formacie PDF zawierający pełną treść niniejszej pracy inżynierskiej.

3. Wymagania techniczne

Do uruchomienia projektu wymagane jest środowisko Unity w wersji 6000.2 lub nowszej.

Specjalność: Front-End Developer

Streszczenie pracy dyplomowej

Wykorzystanie sztucznej inteligencji w programowaniu postaci niezależnych w grach komputerowych

Autor: Bartosz Krawiś

Promotor: Dr Inż. Piotr Bobiński

Słowa kluczowe: sztuczna inteligencja, gry komputerowe, postacie niezależne (NPC), skończone automaty stanów, Unity, platformówka 2D, drzewa behawioralne

Przedmiotem niniejszej pracy inżynierskiej jest projekt oraz implementacja systemu sztucznej inteligencji sterującego zachowaniem postaci niezależnych (NPC) w dwuwymiarowej grze platformowej. Głównym celem pracy było opracowanie oraz analiza mechanizmów odpowiedzialnych za czytelne i przewidywalne zachowanie przeciwników, zrealizowanych w oparciu o skończone automaty stanów (Finite-State Machine). Część teoretyczna obejmuje przegląd definicji i rozwoju sztucznej inteligencji w grach, stosowanych technologii oraz modeli decyzyjnych, w tym maszyn stanów, drzew behawioralnych, systemów regułowych, logiki rozmytej, algorytmów planowania ścieżek oraz uczenia maszynowego, a także zagadnień personalizacji zachowań i aspektów etyczno-prawnych. Część praktyczną zrealizowano w silniku Unity z wykorzystaniem języka C#. Zaimplementowano dwa typy przeciwników — patrolującego oraz strażnika — działających w oparciu o odrębne automaty stanów oraz architekturę komponentową rozdzielającą logikę decyzyjną, obsługę punktów życia i warstwę animacji. Przeprowadzone testy funkcjonalne pozwoliły ocenić poprawność i stabilność działania systemu, w tym skuteczność mechanizmu dwóch progów wykrywania, który wyeliminował niestabilne przełączanie stanów w sytuacjach granicznych. Wyniki potwierdziły, że odpowiednio zaprojektowany system oparty na automatach stanów jest rozwiązaniem wystarczającym dla gry o niewielkiej skali, a o powodzeniu implementacji decyduje przede wszystkim jakość integracji logiki z systemem

animacji oraz fizyki silnika. Pracę zakończono wnioskami oraz wskazaniem kierunków dalszego rozwoju systemu sztucznej inteligencji.

Specialization: Front-End Developer

Diploma Thesis Abstract

The use of artificial intelligence in non-player character programming in computer games

Author: Bartosz Krawiś

Supervisor: Ph.D. Eng Piotr Bobiński

Keywords: artificial intelligence, video games, non-player characters (NPC), finite-state machines, Unity, 2D platformer, behaviour trees

The subject of this engineering thesis is the design and implementation of an artificial intelligence system controlling the behaviour of non-player characters (NPCs) in a two-dimensional platform game. The main objective was to develop and analyse the mechanisms responsible for clear and predictable enemy behaviour, implemented using finite-state machines (FSM). The theoretical part reviews the definition and development of artificial intelligence in games, the technologies used, and decision-making models, including finite-state machines, behaviour trees, rule-based systems, fuzzy logic, pathfinding algorithms and machine learning, as well as issues of behaviour personalisation and ethical and legal aspects. The practical part was carried out in the Unity engine using the C# language. Two types of enemies were implemented — a patrolling enemy and a guard — operating on separate state machines and a component-based architecture that separates decision logic, health handling and the animation layer. The functional tests carried out made it possible to assess the correctness and stability of the system, including the effectiveness of the two-threshold detection mechanism, which eliminated unstable state switching in borderline situations. The results confirmed that a properly designed system based on finite-state machines is a sufficient solution for a small-scale game, and that the success of the implementation depends primarily on the quality of the integration between the logic and the engine's animation and physics systems. The thesis concludes with findings and directions for further development of the artificial intelligence system.